

Multi-dimensional Data Structures

Hoa Dinh Nguyen, PhD.

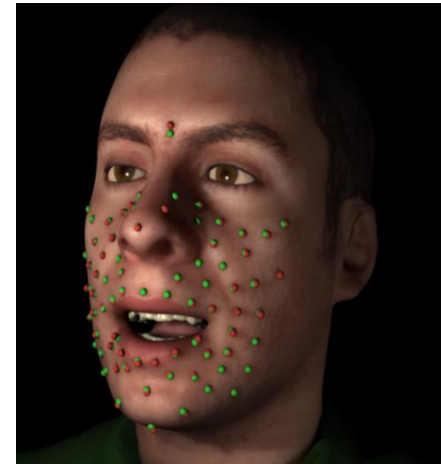
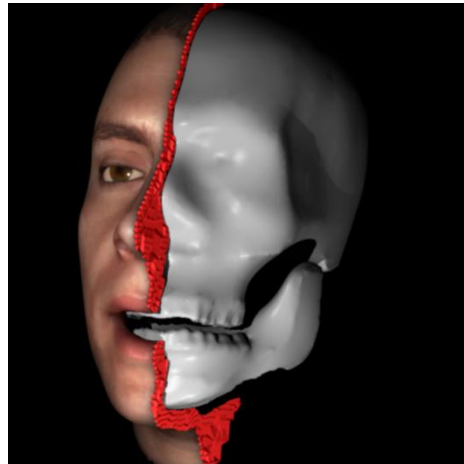
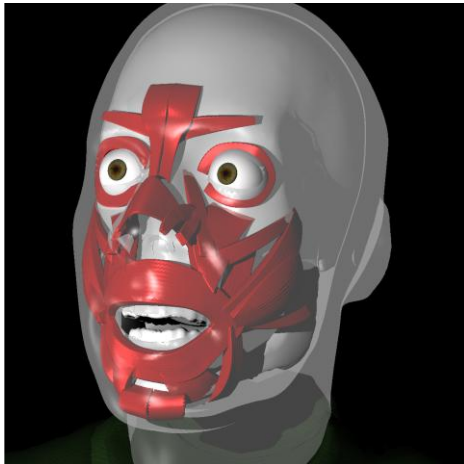
hoand@ptit.edu.vn

Multi-dimensional data structures

- ❑ A geographic information system (GIS) stores information about some physical region of the world.
- ❑ A map is just viewed as a 2-d image, and certain “points” on the map are considered to be interest.
- ❑ Specialized data structures:
 - ▶ k-d trees
 - ▶ Point Quadrees
 - ▶ MX-Quadrees
 - ▶ PR-Quadrees
 - ▶ R-Trees

Multi-dimensional data structures

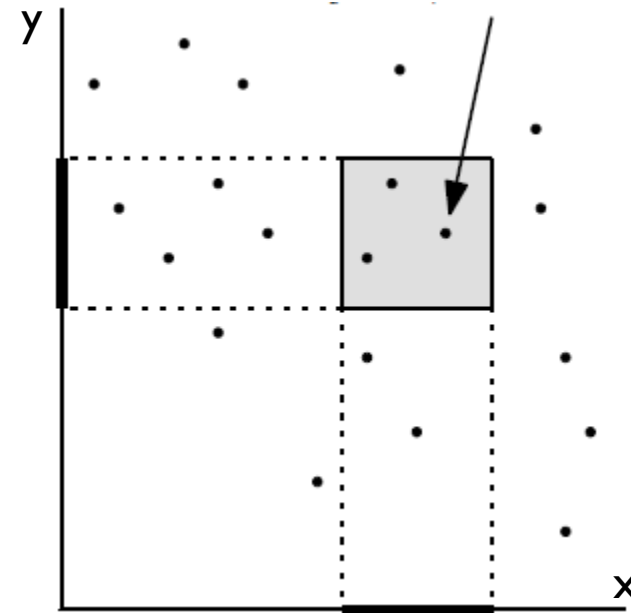
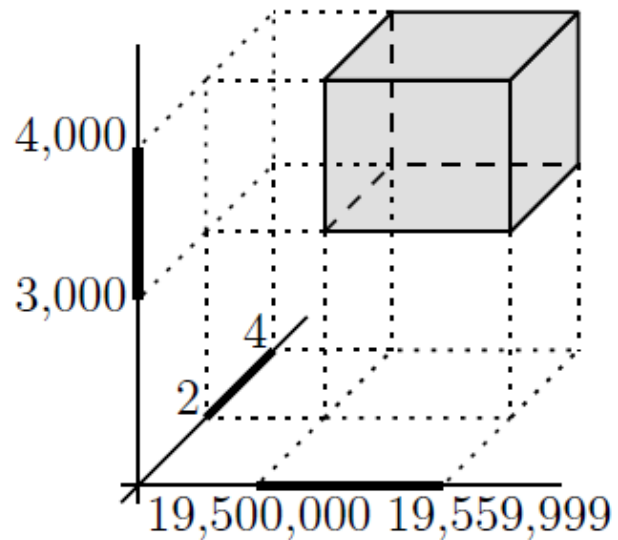
- ▶ Applications:
 - ▶ Data storage and recovery (spatial)
 - ▶ Data resurfacing
 - ▶ Machine learning applications
 - ▶ Intelligent interactive technologies
 - ▶ Optimization



Multi-dimensional data structures

► Data queries

- Accurate matching: Search for matched data by all attributes
- Similarity matching: Correct data but not on matching attribute values
- Matching according to intervals



Spatial Queries in Multimedia and GIS

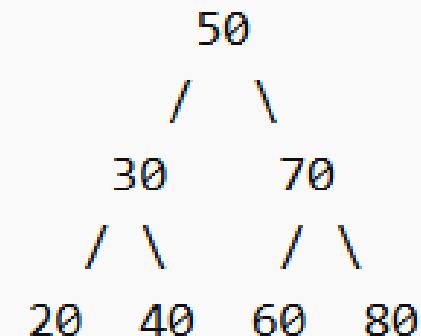
- ▶ Exact match: all attributes equal — use B-tree / hash on composite key (limited dimension)
- ▶ Range query: find points where $x \in [a,b]$ AND $y \in [c,d]$ — core GIS operation
- ▶ Nearest neighbor (NN): closest point to query q — CBIR, k-NN classification
- ▶ k-NN: k closest points — vector similarity (Lecture 7)
- ▶ Window / region query: all objects intersecting a rectangle or polygon
- ▶ Similarity matching: nearest neighbor in feature space, not exact attribute match



Binary data storage structure for 1-dimensional data

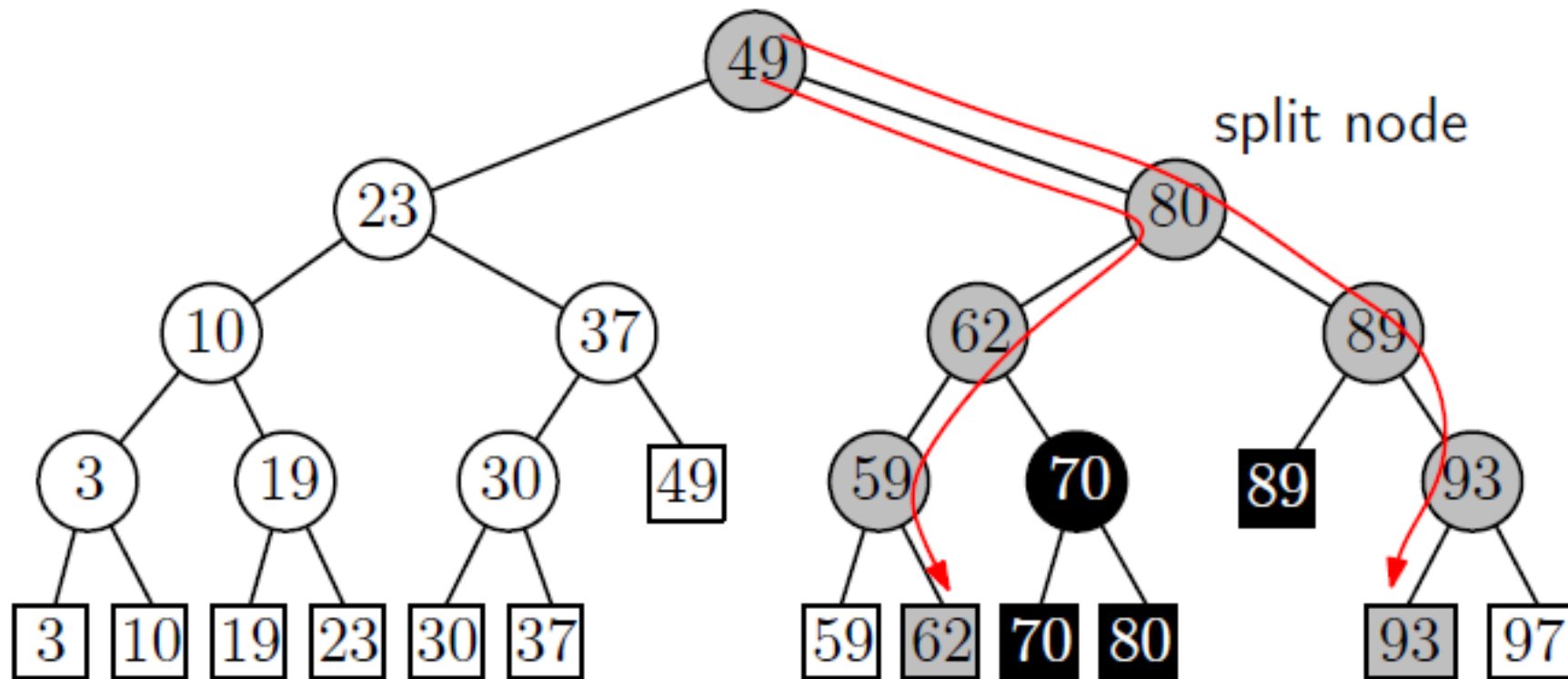
- ▶ The data is presented as a linear collection (single key per element), but the storage organization is hierarchical.
- ▶ Each node contains:
 - ▶ A data value
 - ▶ A pointer to the left child
 - ▶ A pointer to the right child
- ▶ Each node has at most two children
- ▶ Even though the structure is hierarchical, the stored data itself can represent a 1-dimensional dataset (e.g., list of numbers).
- ▶ Example:

Binary tree for the data [50, 30, 70, 20, 40, 60, 80]:



Binary data storage structure for 1-dimensional data

- ▶ Example: find all values in the range [61, 90]



Multi-dimensional data structures

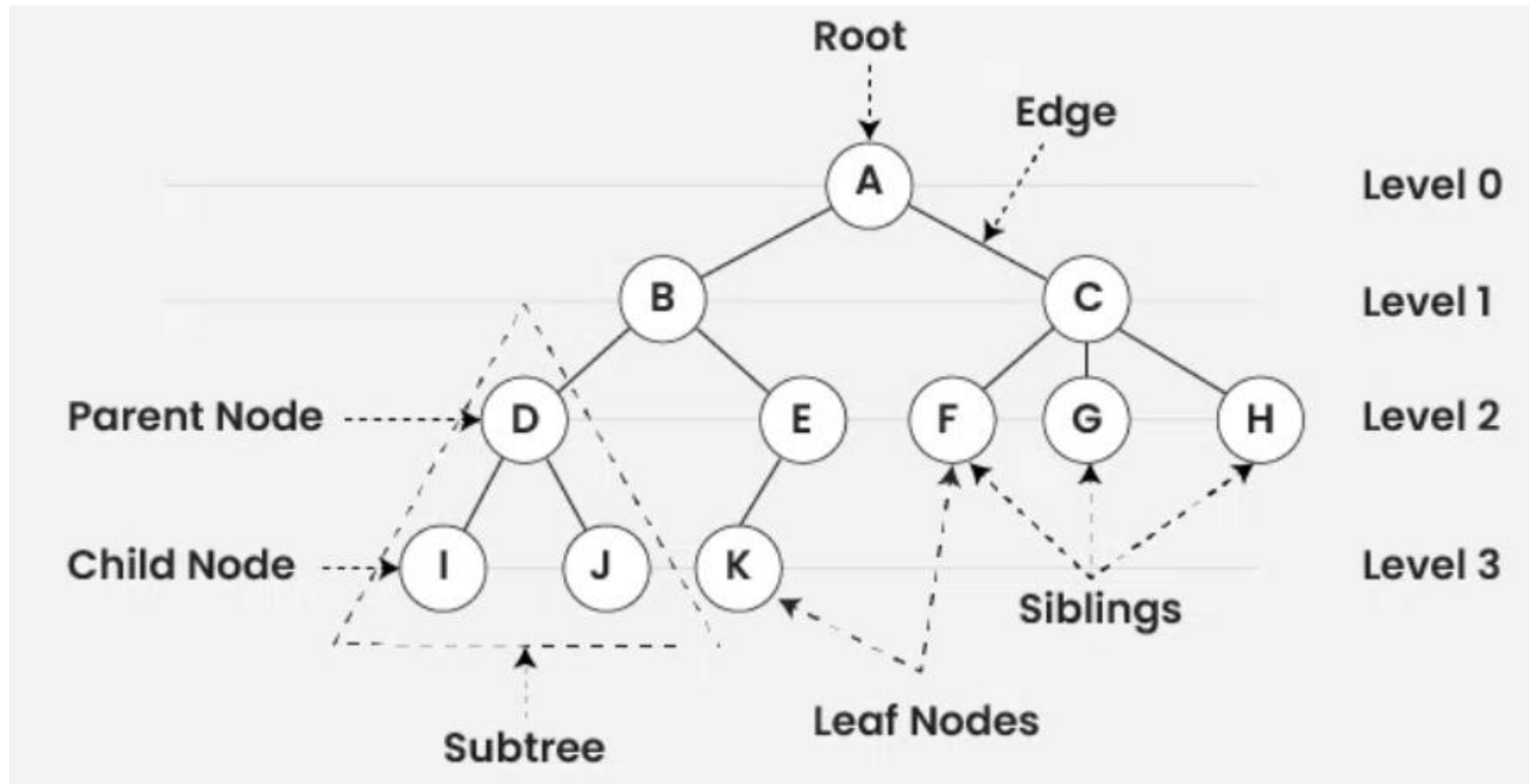
- ▶ Ideas:

- ▶ Construct binary trees according to each attribute (dimension) of the data
- ▶ Perform a data lookup according to each attribute (dimension) that we find most effective (find the fastest results).

From 1D BST to Multidimensional Trees

- ▶ BST: one key per node; left < root < right — $O(\log n)$ search if balanced
- ▶ Range query [61,90]: traverse BST, prune subtrees outside range
- ▶ Extension to k dimensions: cycle splitting dimension by level (k-d tree)
- ▶ Alternative: divide space into 4 quadrants per node (quadtree)
- ▶ Curse of dimensionality: in high k , all points equidistant — trees less effective
- ▶ For MM: 2D–5D common (geo, color, feature vectors); use specialized structures

Binary structures for Multi-dimensional data



k-d trees

▶ Concept

- ▶ Used to store k dimensional point data.
- ▶ As a tree that groups data spatially using the recursive method
- ▶ Each node is a (structured) data record
- ▶ Data is split hierarchically based on one-by-one attribute axis of data
- ▶ Each data node stores at least one subsequent node on each attribute axis.
- ▶ A 2-d tree stores 2-dimensional point data; a 3-d tree stores 3-dimensional point data, and so on.

Construction of k-d trees

- ▶ A 2-d tree is a binary tree satisfying the following condition (with root a level 0)
 - ▶ For node N with level(N) is even, then every node M under N.LLINK has the property that $M.XVAL < N.XVAL$, and every node P under N.RLINK has the property that $P.XVAL \geq N.XVAL$.
 - ▶ For node N with level(N) is odd, then every node M under N.LLINK has the property that $M.YVAL < N.YVAL$, and every node P under N.RLINK has the property that $P.YVAL \geq N.YVAL$.
- ▶ Expansion for k-d trees with $k > 2$: The order of dividing the coordinate axis varies with each level of the data tree.
- ▶ Example: 2-d tree, odd level divided by x, even level divided by y.
3-d tree: $x \rightarrow y \rightarrow z \rightarrow x \rightarrow y \dots$

k-d Tree Construction — Algorithm Summary

- ▶ Input: set of k-dimensional points; splitting dimension $d = \text{level} \bmod k$
- ▶ Choose root (first inserted point in slide examples; median split in balanced variants)
- ▶ At level ℓ : split on $d = \ell \bmod k$; left subtree $<$ pivot on d ; right subtree $\geq$ pivot
- ▶ 2D data: even level $\rightarrow x$, odd level $\rightarrow y$
- ▶ Build recursively; balance improves if median split used
- ▶ Storage: each node stores point + left/right pointers (LLINK / RLINK)



Example 2-d trees

INFO (XVAL, YVAL)

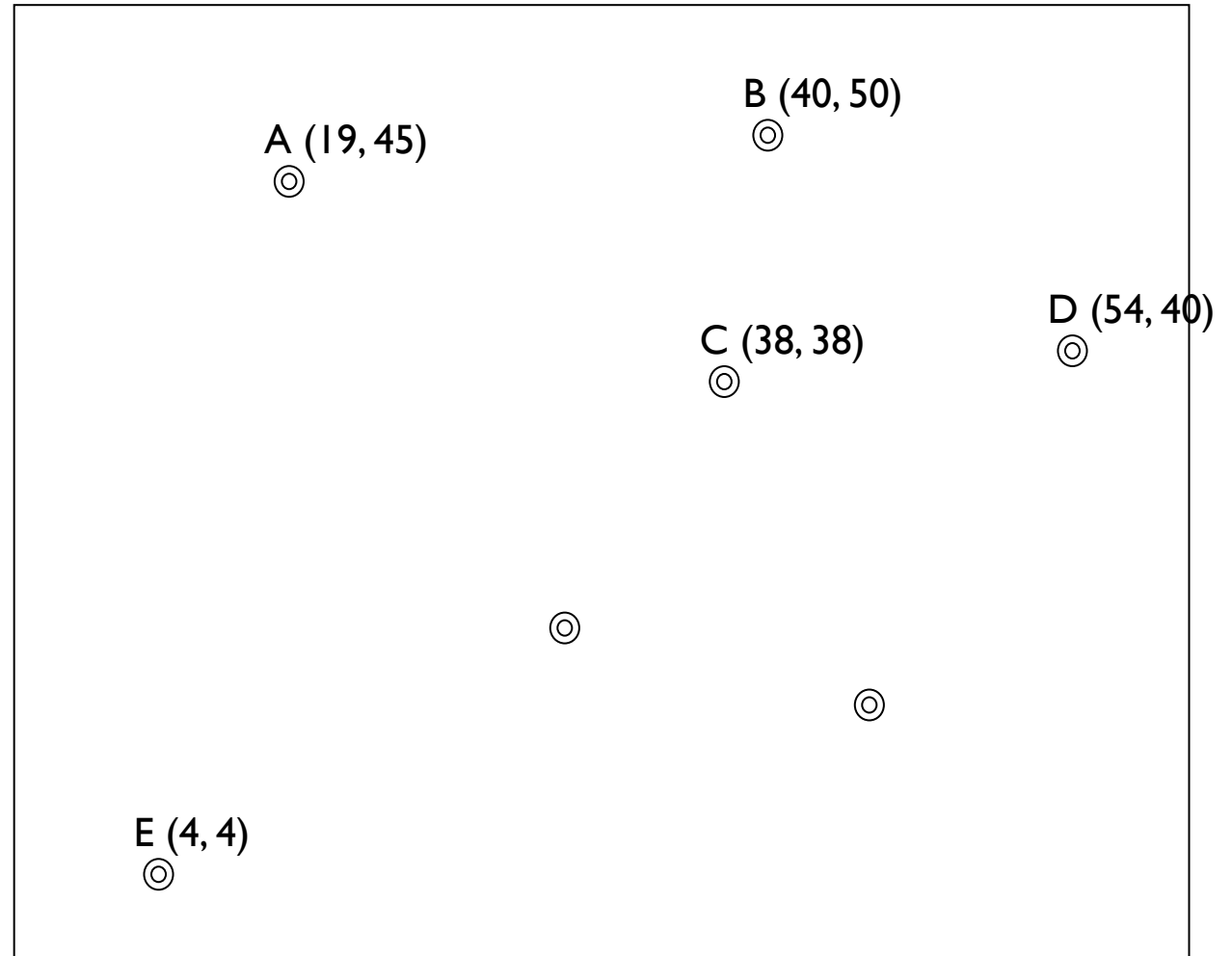
A (19, 45)

B (40, 50)

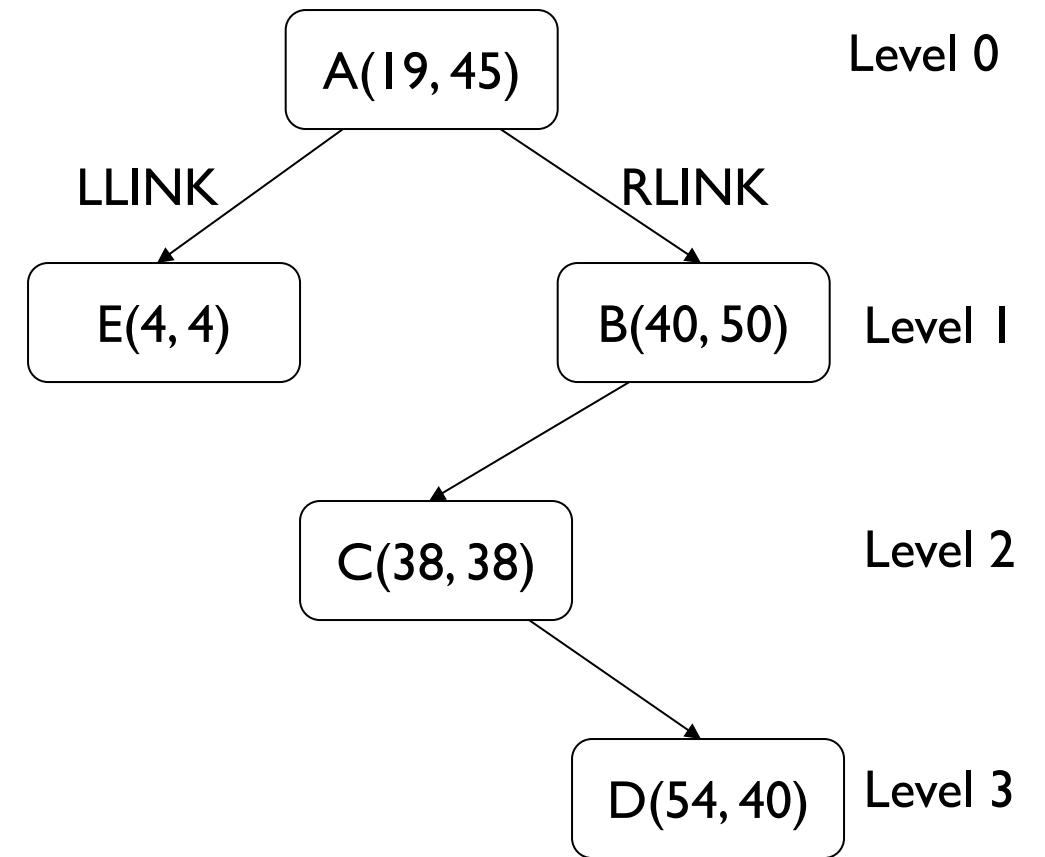
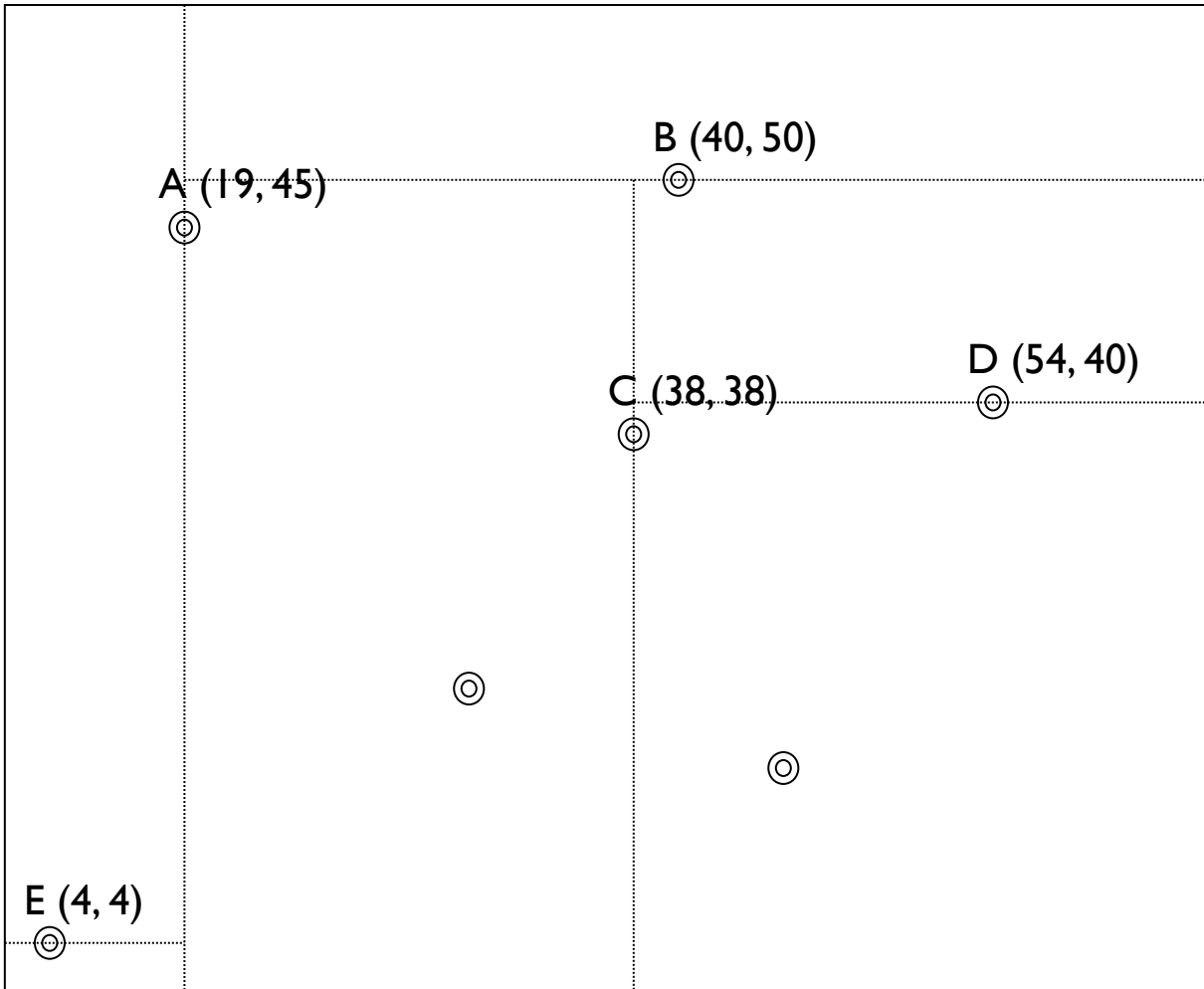
C (38, 38)

D (54, 40)

E (4, 4)



Example 2-d trees

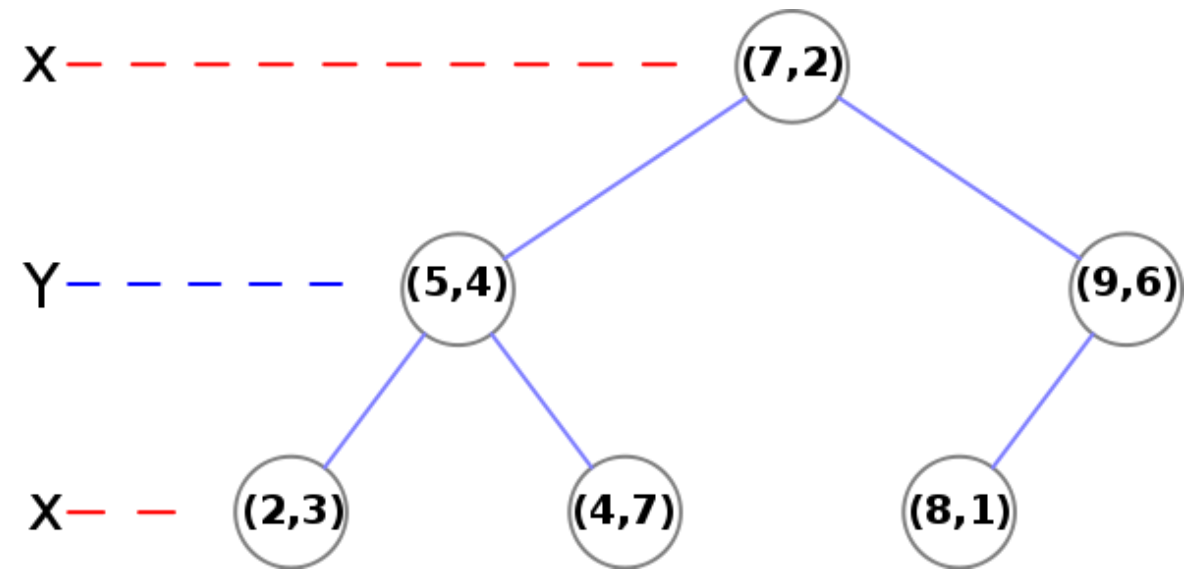
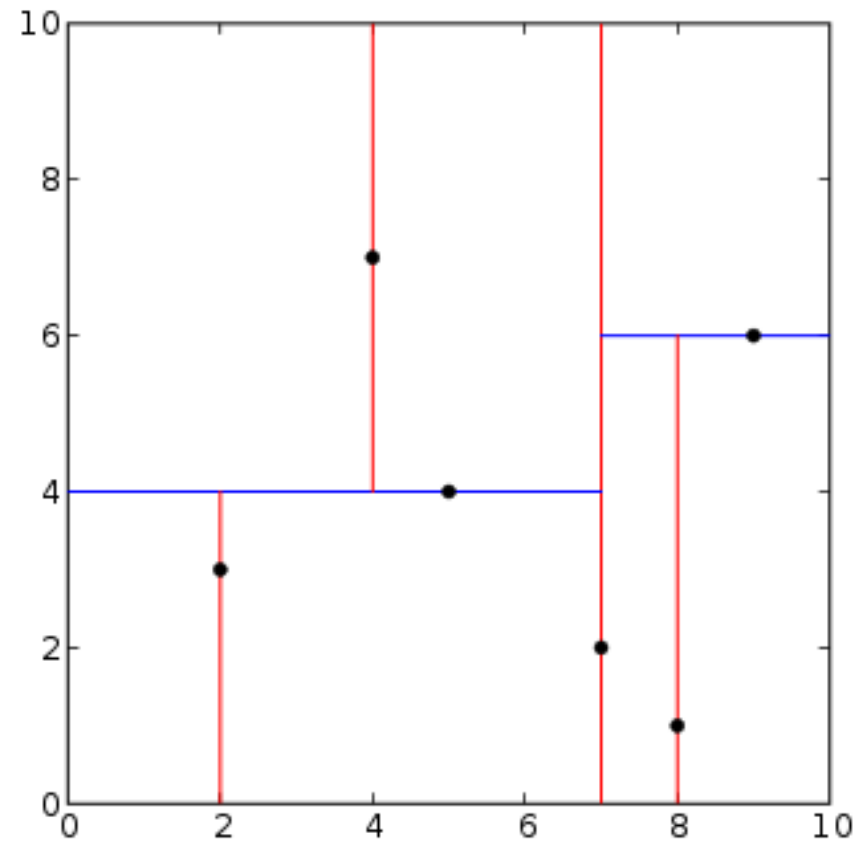


2-d Tree Spatial Partition (Diagram Guide)

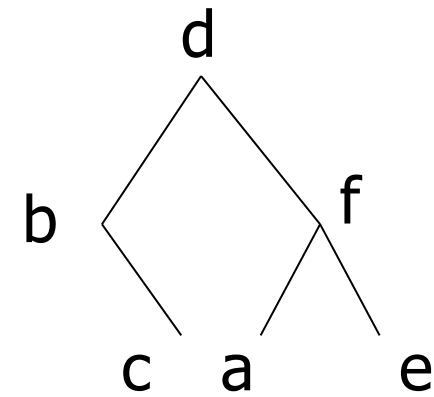
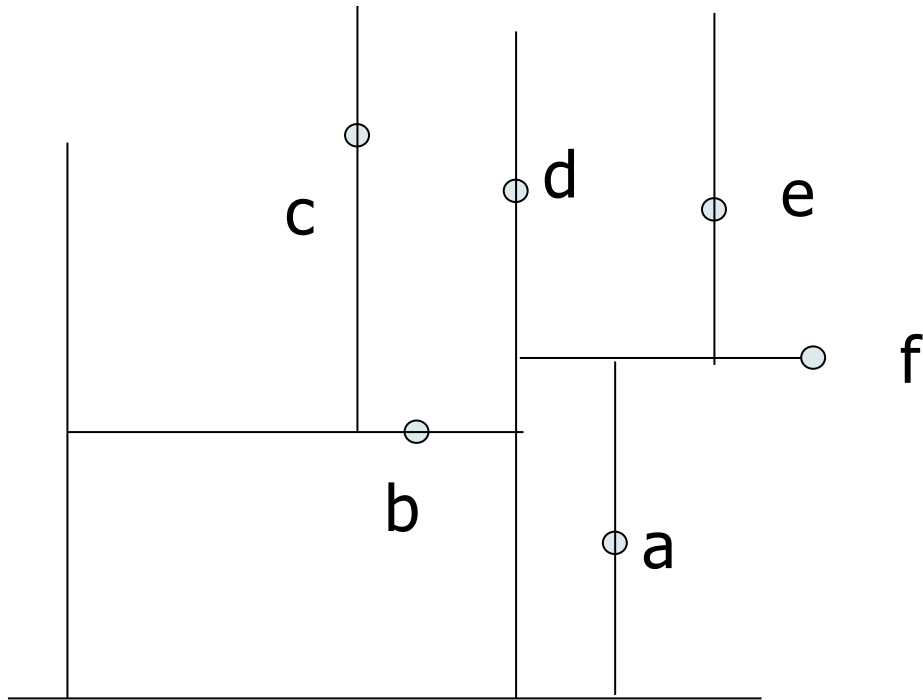
- ▶ Insert order A,D,C,B,E $\rightarrow$ root A(19,45) splits on x at level 0
- ▶ Left of A: points with $x < 19 \rightarrow$ E(4,4); right: D(54,40), C(38,38), B(40,50)
- ▶ At D (level 1, y-split): C(38,38) left ($y < 40$), B(40,50) right ($y \geq 40$)
- ▶ Vertical lines = x-splits; horizontal lines = y-splits (see slide 14 labels a–f)
- ▶ Use this partition when drawing the tree in Exercise 2
- ▶ Compare with slide 12 tree diagram: same points, explicit level labels



Example 2-d trees



Example 2-d trees



Addition in k-d trees

- ▶ Adding a node in the 2-d tree:
 - ▶ Starting from the root node (level 0): add the child node to the left or right along the x-axis based on the x value of the child node
 - ▶ To level 1 node: add the child node to the left or right side along the y-axis
 - ▶ To level 2 node: add the child node to the left or right side along the x-axis
 - ▶
- ▶ Expansion to 3-d, k-d trees

Deletion in 2-d Trees

Suppose T is a 2-d tree, and we wish to delete $N(x, y)$.

❑ If N is a leaf node, then set link to NIL and delete N .

❑ Otherwise,

Step 1: Find a “candidate replacement” node R that occurs either in T_i for $i \in \{\ell, r\}$. (T_i under N)

Step 2: Replace all of N 's non-link fields by those of R .

Step 3: Recursively delete R from T_i .

Candidate Replacement Node

- ❑ If we find a replacement node from left subtree, we might violate the definition of 2-d tree. (Two nodes have equal maximal values)
- ❑ Set priority to find the replacement node R from right subtree (T_r).
 - ▶ Every node M in T_r is such that $M.XVAL \geq R.XVAL$ if $level(N)$ is even, and $M.YVAL \geq R.YVAL$ if $level(N)$ is odd.
 - ▶ If T_r is not empty, and $level(N)$ is even (odd), then any node in T_r that has the smallest possible $XVAL$ ($YVAL$) field in T_r is a candidate replacement node.

Candidate Replacement Node

□ If T_r is empty –

- ▶ Find the node R' in left subtree T_ℓ with the largest $XVAL$ if $level(N)$ is even, or the largest $YVAL$ if $level(N)$ is odd.
- ▶ Replace all of N 's nonlinked fields by those of R' .
- ▶ Set $N.RLINK = N.LLINK$ and set $N.LLINK = NIL$.
- ▶ Recursively delete R' .

k -d Tree for $k \geq 2$

For each node N

- Suppose $level(N) \bmod k = i$.
- For each node M in N 's left subtree, $M.VAL[i] < N.VAL[i]$.
- For each node P in N 's right subtree, $P.VAL[i] \geq N.VAL[i]$.

Note: when $k=1$, we get a standard binary search tree.

Variations of k-d trees

- ▶ Variations of the structure of the 2-d tree:
 - ▶ Split child nodes (left, right) based on the average value of each attribute
 - ▶ Divide subnodes (left, right) based on the median of each attribute.
- ▶ Expansion to k-d ($k > 2$): each level is divided according to each attribute, sequentially selecting the attributes according to each level.

Data queries in k-d trees

- ▶ Search for data by searching route containing points closest to the query
- ▶ 2 common types of lookups:
 - ▶ Nearest neighbor search (NN)
 - ▶ k-nearest neighbor search (k-NN)

Nearest Neighbor Search on k-d Trees

- ▶ 1. Traverse from root to leaf following query q (same path as insert)
- ▶ 2. Leaf = current best candidate; $\text{dist_best} = d(q, \text{leaf})$
- ▶ 3. Backtrack: if $|q[d] - \text{pivot}[d]| < \text{dist_best}$, search other subtree
- ▶ 4. Update dist_best when a closer point is found
- ▶ k-NN: maintain max-heap of k best; prune when plane distance $>$ worst of k
- ▶ Range search: recurse only into subtrees whose regions intersect query box



NN search in k-d trees

- ▶ Starting at the root node, work your way down to the child nodes (left or right) so that stored nodes are close to the query data
- ▶ Once you've reached the top node, select the one lying on the route that is closest to the query as the "best result"
- ▶ Repeat the procedure corresponding to multiple possible routes to find "best of the best" result.
- ▶ The algorithm can follow both directions (left and right) of the k-d tree if the dividing plane at the root node intersects with the range of values to be found.

K-NN search in k-d trees

- ▶ Same as NN search
- ▶ Select k best results
- ▶ Range search:
 - ▶ Search for points with each attribute within each defined interval.

Point Quadrees

- ▶ Same as k-d trees for the way to divide data points into branches.
- ▶ At each node, the space is divided into 4 parts according to 2 planes perpendicular to each pair of coordinate axes (NW, SW, NE, SE)
- ▶ Each quartile corresponds to a child of that node
- ▶ Each node has up to 4 child nodes
- ▶ In multidimensional data, the division of the data domain is done based on the sequential sequence of each pair of coordinate axes
 - ▶ Example: 3-dimensional data (X, Y, Z): The data is divided based on pairs of coordinate axes $(x, y) \rightarrow (y, z) \rightarrow (z, x) \rightarrow (x, y) \dots$

Point Quadrees

- ❑ XLB , XUB , YLB , and YUB are upper bounds and lower bounds of X and Y , respectively.
- ❑ Suppose N is a root node of a tree or subtree of quadtree:
 - ▶ Every node P_1 in $N.NW$ is such that $P_1.XVAL < N.XVAL$ and $P_1.YVAL \geq N.YVAL$.
 - ▶ Every node P_2 in $N.SW$ is such that $P_2.XVAL < N.XVAL$ and $P_2.YVAL < N.YVAL$.
 - ▶ Every node P_3 in $N.NE$ is such that $P_3.XVAL \geq N.XVAL$ and $P_3.YVAL \geq N.YVAL$.
 - ▶ Every node P_4 in $N.SE$ is such that $P_4.XVAL \geq N.XVAL$ and $P_4.YVAL < N.YVAL$.

Point Quadtree structure

□ Node structure in a point quadtree:

```
qtnodetype = record
```

INFO: infotype;

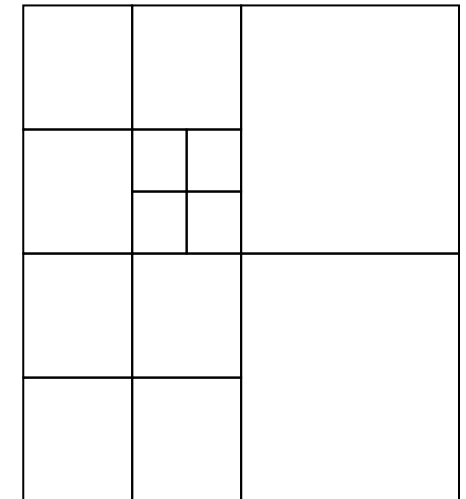
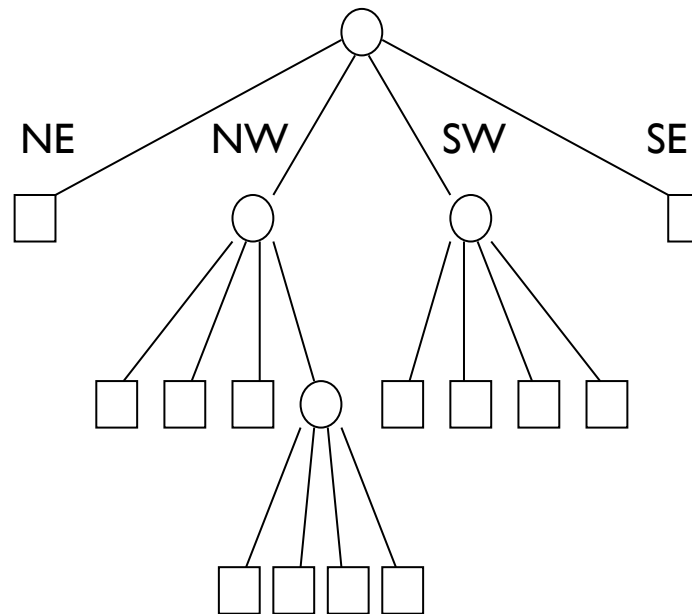
XVAL: real;

YVAL: real;

$$\text{XLB, XUB, YLB, YUB: } \text{real} \cup \{+\infty, -\infty\}$$

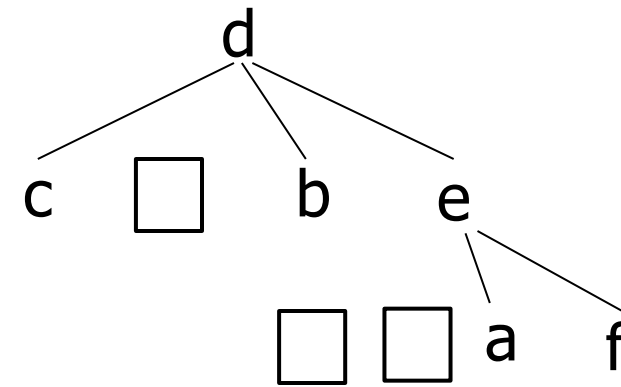
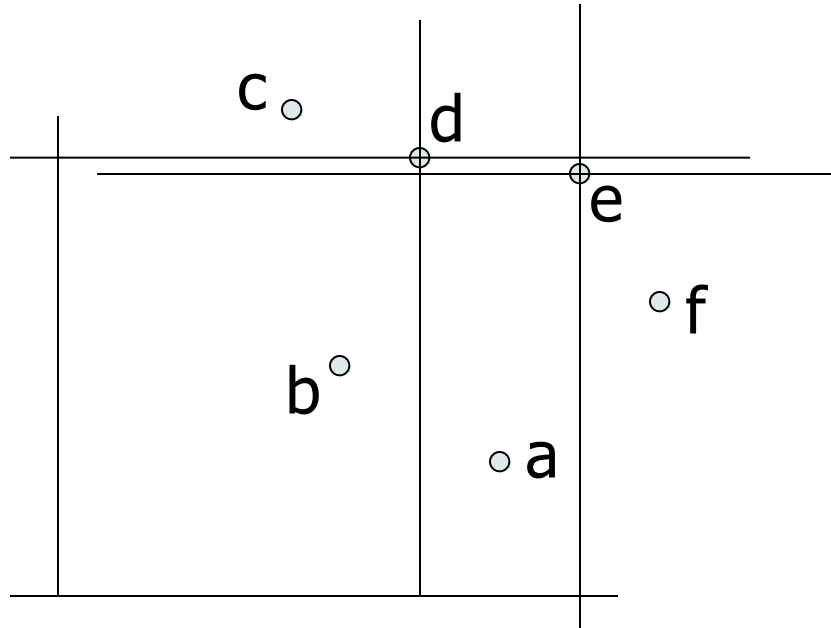
```
NW, SW, NE, SE: ↑qtnodetype;
```

end



Point Quadtree structure

► Example:



Point Quadtree structure

INFO (XVAL, YVAL)

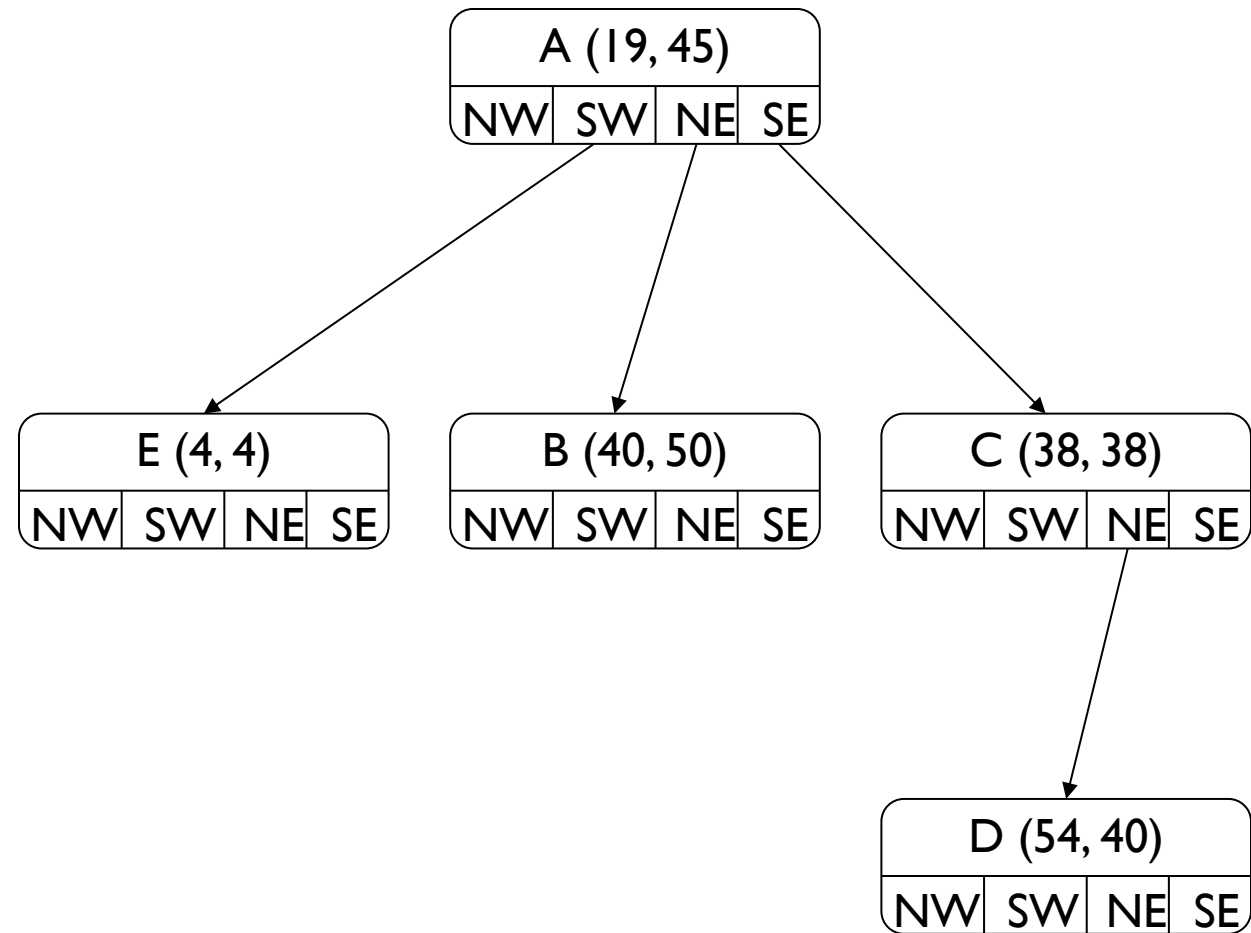
A (19, 45)

B (40, 50)

C (38, 38)

D (54, 40)

E (4, 4)



Point Quadtree structure

INFO (XVAL, YVAL)

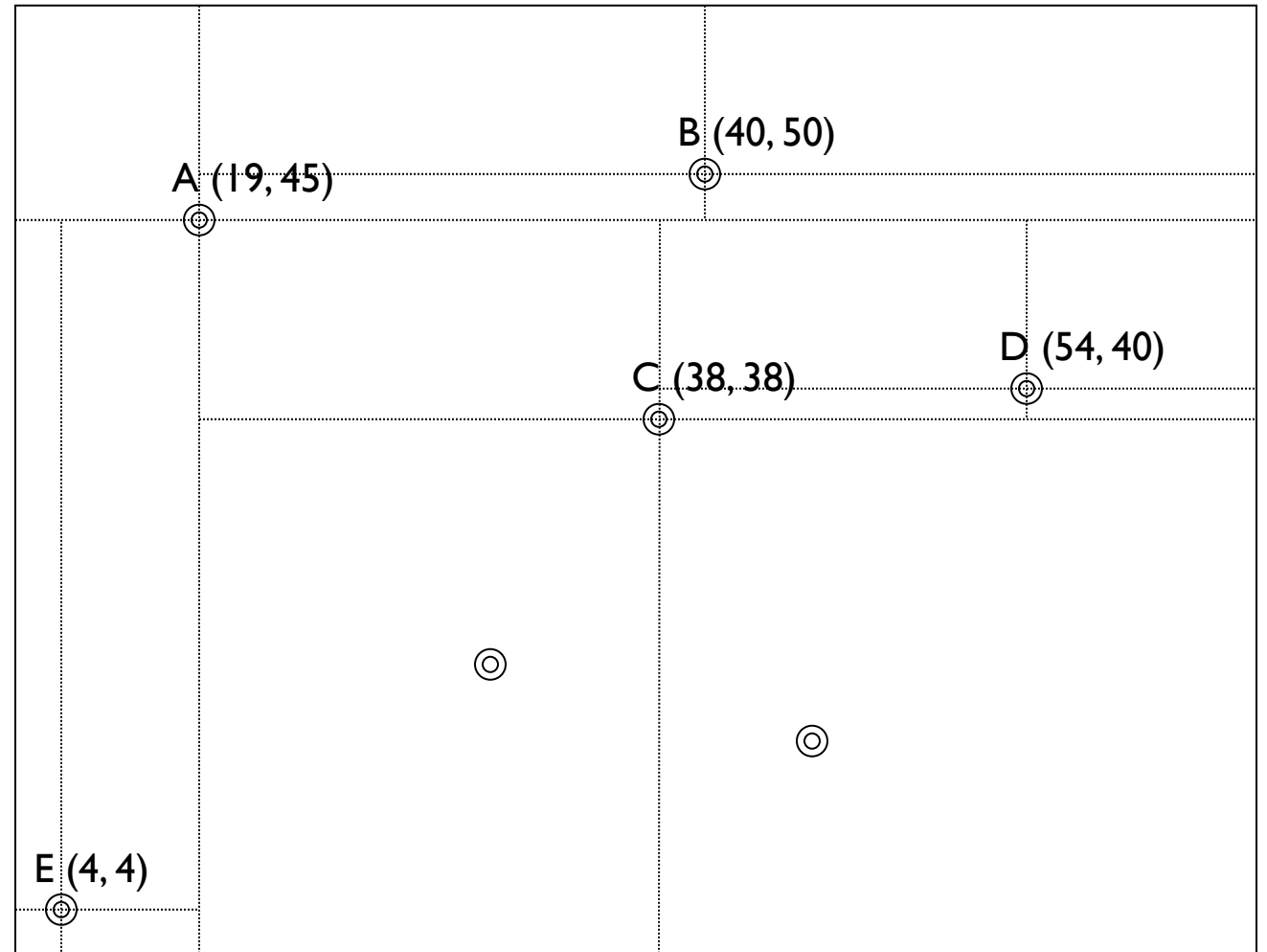
A (19, 45)

B (40, 50)

C (38, 38)

D (54, 40)

E (4, 4)



Deletion in Point Quadrees

A node N to be deleted --

- ❑ If N is a leaf node, the link from parent to N is set to NIL then releases node N .
- ❑ Otherwise, try to find a replacement node R from subtrees such that:
 - ▶ Every other node R' in $N.NW$ ($N.SW / N.NE / N.SE$) is to the north-west (south-west / north-east / south-east) of R .
 - ▶ Unfortunately, replacement node may not be found. (i.e., deleting A)
 - ▶ Thus, in the worst case, deletion of an interior node N may require reinsertion of some nodes in the subtrees pointed to by $N.NE$, $N.SE$, $N.NW$, and $N.SW$.

Range queries in Point Quadrees

- ❑ Do not search regions that do not intersect the circle defined by the query.
- ❑ Search procedure:

```
ProcRangeQueryPQtree(T:newqtnodetype, C:circle)
  if region(T)  $\cap$  C =  $\emptyset$  then Halt
  else
    if (T.XVAL, T.YVAL)  $\in$  C then print (T.XVAL, T.YVAL);
    ProcRangeQueryPQtree(T.NW, C);
    ProcRangeQueryPQtree(T.SW, C);
    ProcRangeQueryPQtree(T.NE, C);
    ProcRangeQueryPQtree(T.SE, C);
  end proc
```

Characteristics of k-d trees and Point Quadrees

- ▶ The shape and structure of the tree depends on the order of selection of the nodes in the tree
- ▶ Each choice affects the process of entering and looking up information.
- ▶ At each node, the data space is divided into 2 parts (k-d trees) or 4 parts (point quadrees).
- ▶ Data nodes are evenly or unevenly divided into branches.

k-d Trees vs Point Quadrees — Comparison

- ▶ k-d tree: binary partition; alternating axis; good for exact point location $O(\log n)$
- ▶ Point quadtree: 4-way partition (NW,SW,NE,SE); natural for 2D map layers
- ▶ k-d: simpler for $k > 2$; quadtree intuitive when splits align with quadrants
- ▶ Both degrade with skewed data unless balanced/median splits are used
- ▶ Memory: k-d one node per point; quadtree may have empty quadrants



MX-Quadrees (Matrix Quadrees)

- MX-quadrees attempt to ensure that the shape of the tree are independent of the number of nodes present in the tree, as well as the order of insertion of these nodes.
- MX-quadrees also attempt to provide efficient deletion and search algorithms.
- The map being represented is “split up” into a grid of size $(2^k \times 2^k)$ for some k .
- All physical data are performed at leaf nodes.
- Node Structure: Exactly the same as for point quadrees.

MX-Quadrees (Matrix Quadrees)

- ▶ Store raster/matrix data: each leaf = uniform cell value (color, land type)
- ▶ Fixed recursive 2×2 subdivision until cells homogeneous or max depth reached
- ▶ Internal nodes = mixed region; leaves = attribute value or single point
- ▶ Points stored only at leaf level — deletion may collapse empty parent
- ▶ Different from point quadtree: division by grid cells, not point coordinates
- ▶ Use: satellite images, game maps, occupancy grids



MX-Quadrees

- ▶ MX quadtree construction: (example for 2D data)
 - ▶ The coordinate plane is divided into $2^k \cdot 2^k$ equal parts (fixed k)
 - ▶ The root node represents the entire data domain
 - ▶ Data domains are divided at a central point.
 - ▶ All data nodes are located at level k
- ▶ Expanding to the multidimensional case: splitting the dataspace domain into $2^k \cdot 2^k \dots 2^k$ equal parts, do the same as above.

Example of MX-Quadrees

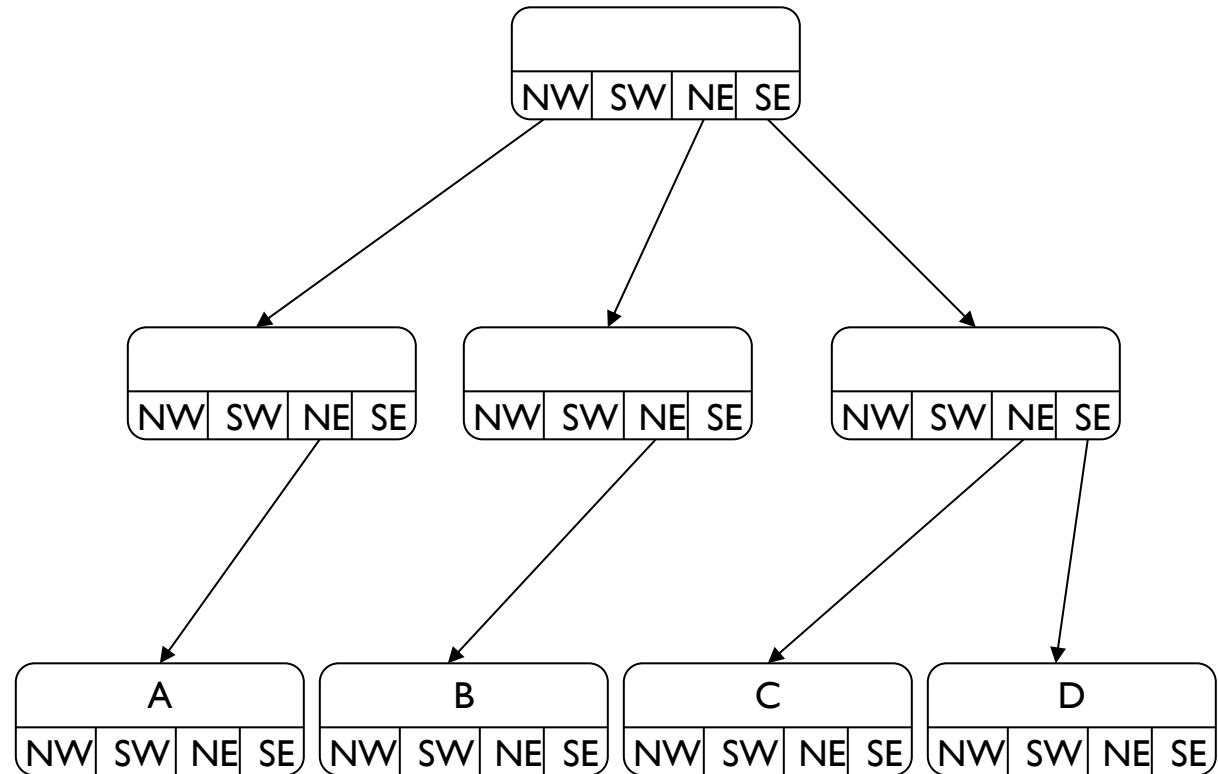
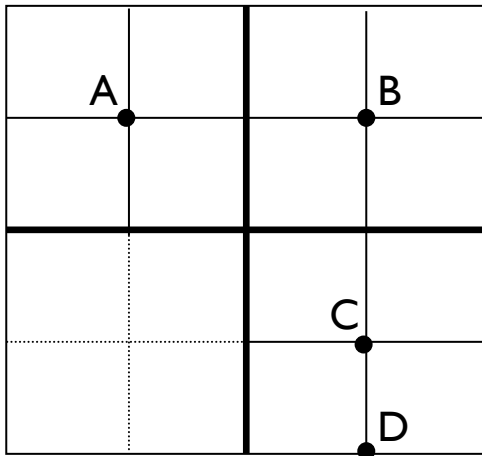
INFO (XVAL, YVAL)

A (1, 3)

B (3, 3)

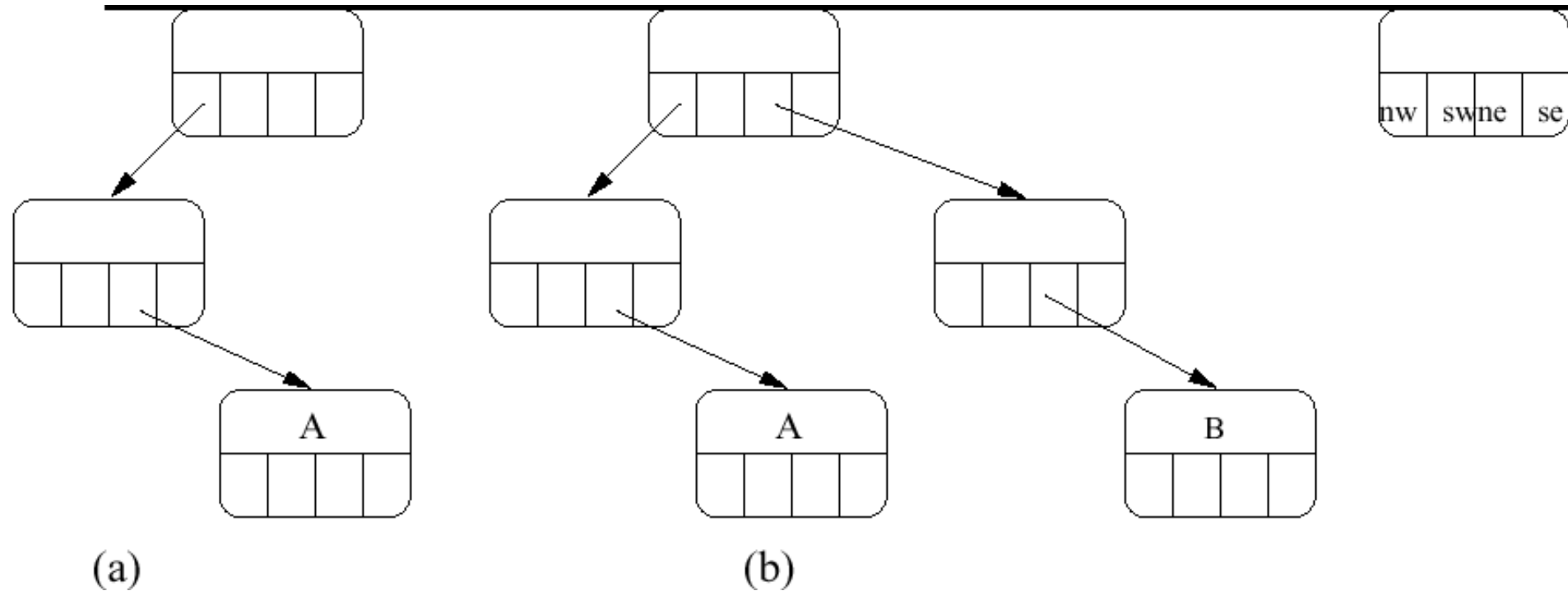
C (3, 1)

D (3, 0)



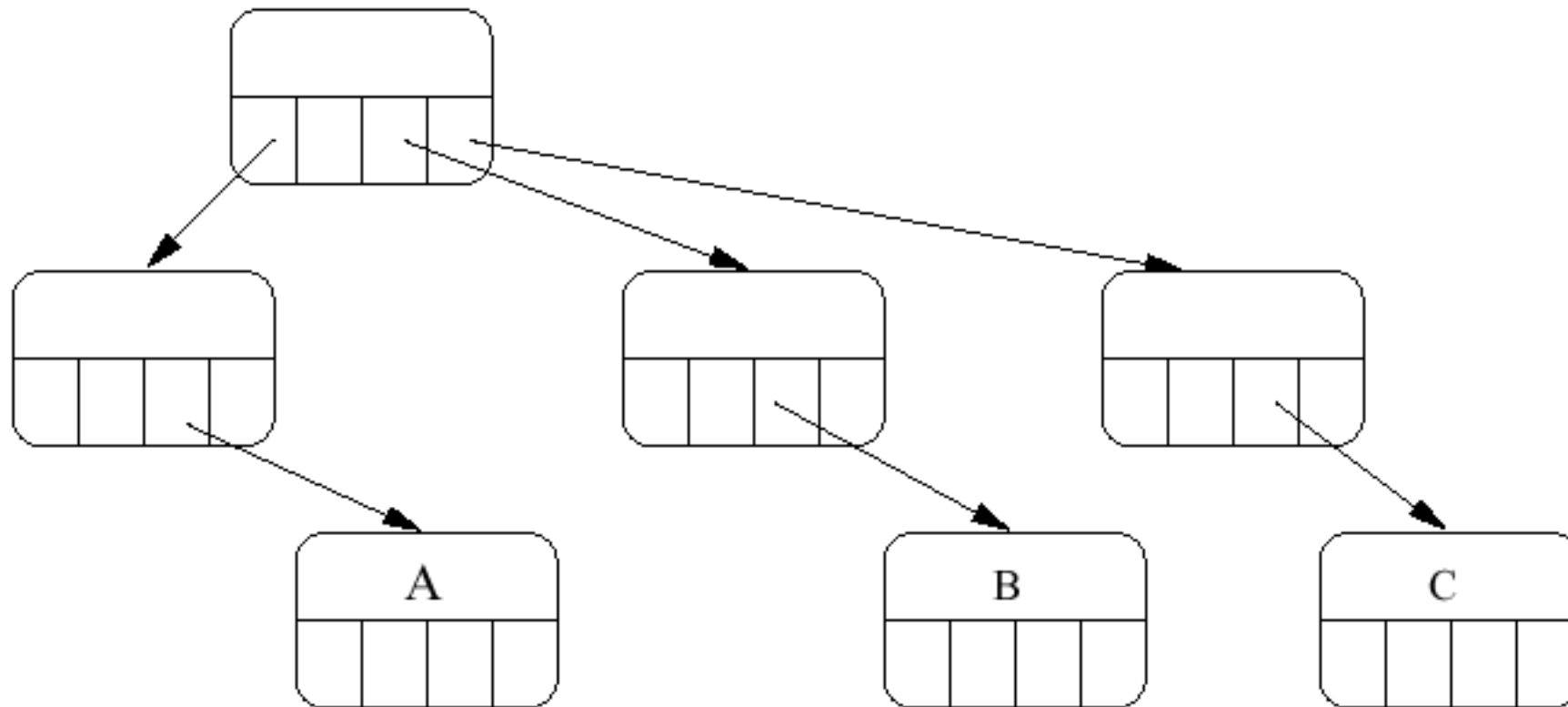
Example of MX-Quadrees

- ▶ Add data nodes A, B to the MX-quadtree



Example of MX-Quadrees

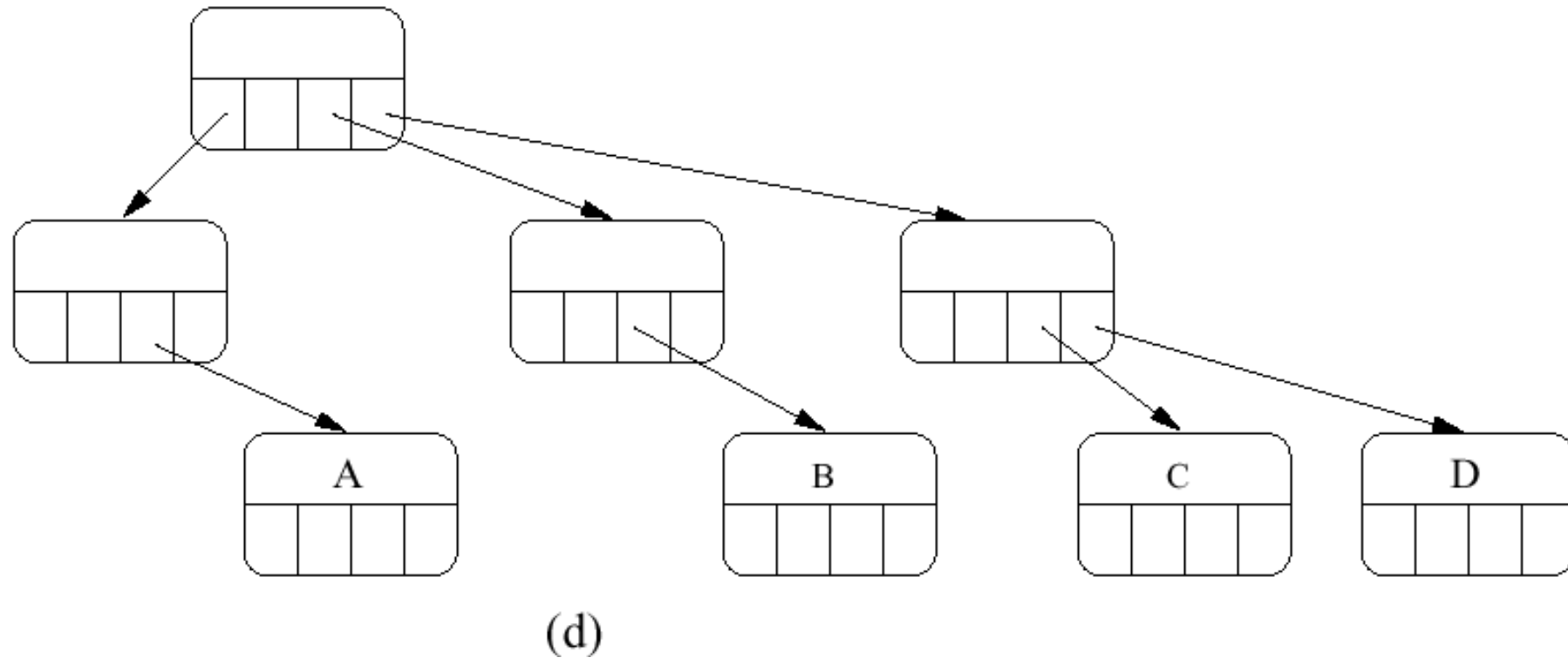
- ▶ Add data node C to MX-quadtree



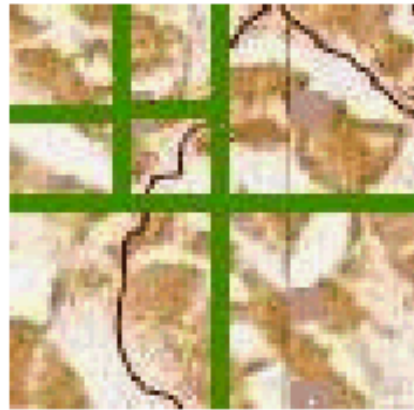
(c)

Example of MX-Quadrees

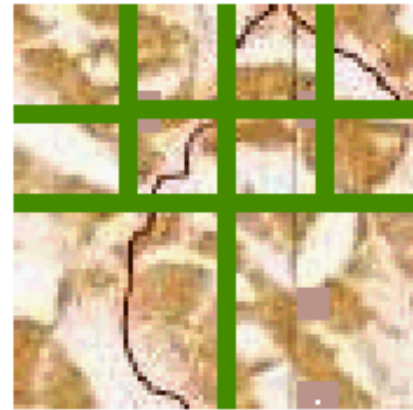
- ▶ Add data node D to the MX-quadtree



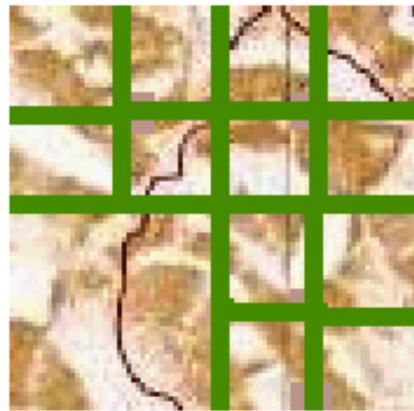
Example of MX-Quadrees



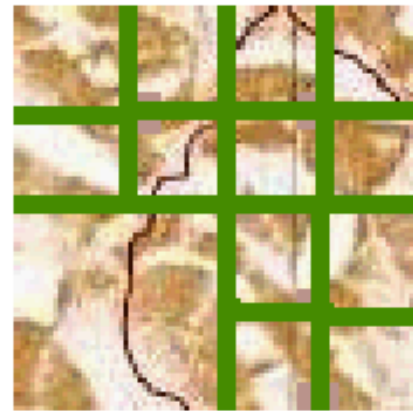
After insertion of A



After Insertion of B

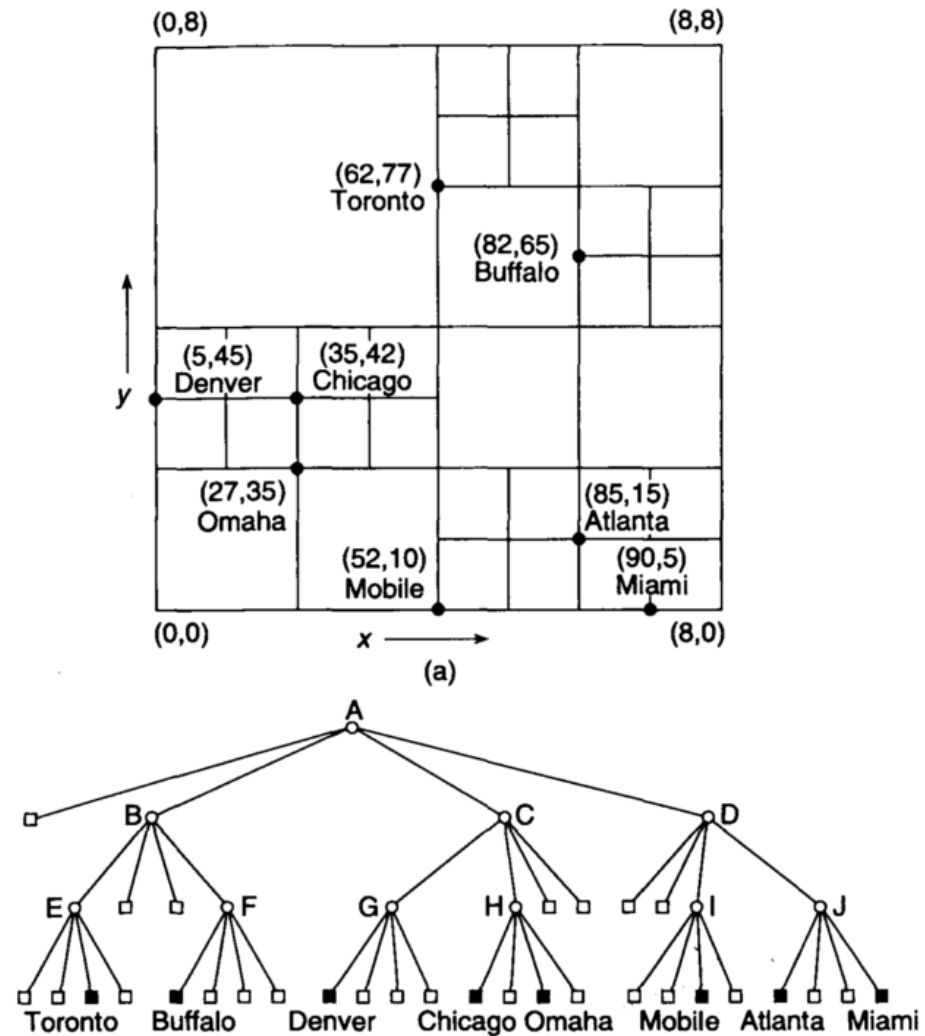


After insertion of C



After Insertion of D

MX Quadtrees - Example



Deletion in MX-Quadrees

- ❑ Deletion in an MX-quadtree is a fairly simple operation, because all points are represented at the leaf level.
- ❑ When a node is deleted, we need to keep trace its parent node to see whether it contains no child node? If so, continue to delete the parent nodes recursively.

Range Queries in MX-Quadrees

- ▶ Handled in exactly the same way as for point quadtrees. But there are two differences:
- ▶ The content of XLB, XUB, YLB, YUB, fields is different from that in the case of point quadtrees.
- ▶ As points are stored at the leaf level, checking to see if a point is in the circle defined by the range query needs to be performed only at the leaf level.

Point-Region quadtrees

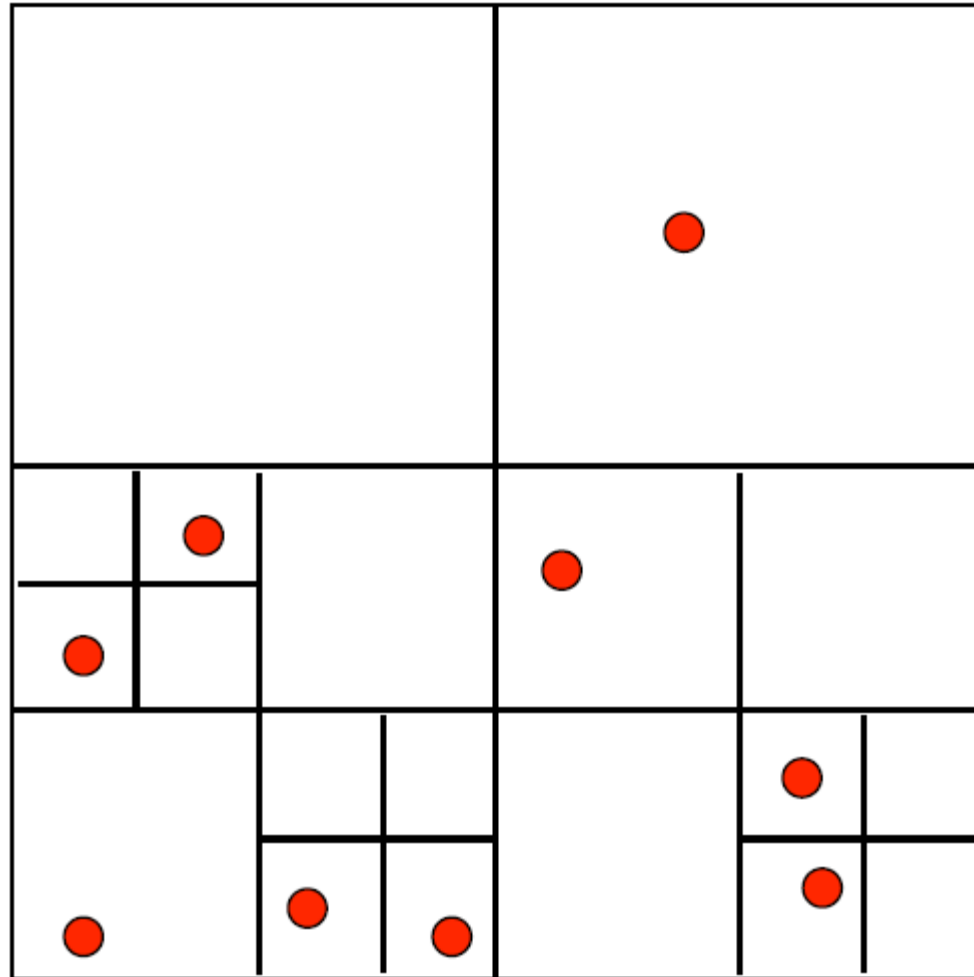
- ▶ Same idea as MX-quadtrees
- ▶ Data domains are divided recursively into 4 parts so that in the end there is only one data node that represents a subdomain.
- ▶ No data node is a child of another node.

PR-Quadrees (Point-Region Quadrees)

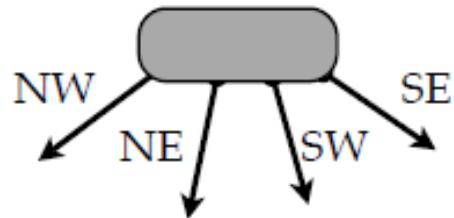
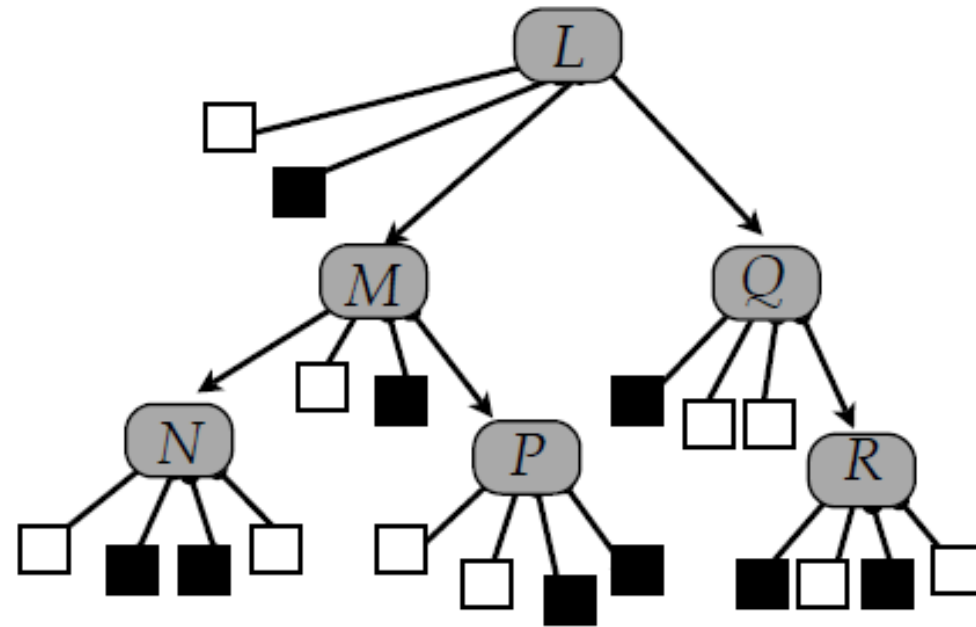
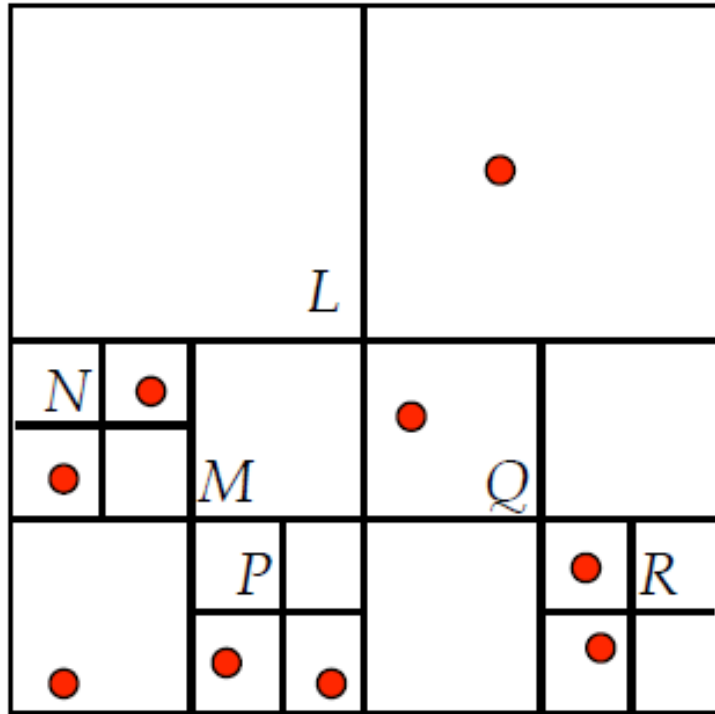
- ▶ Like MX: region subdivision independent of point coordinates
- ▶ Leaf may hold multiple points in same smallest subregion
- ▶ No point node is child of another — all points at leaves
- ▶ Advantage: uniform grid alignment; good for bucketing dense clusters
- ▶ Disadvantage: deep tree when points cluster closely
- ▶ Query: same range traversal as MX; test all points in each leaf bucket



Point-Region quadtrees

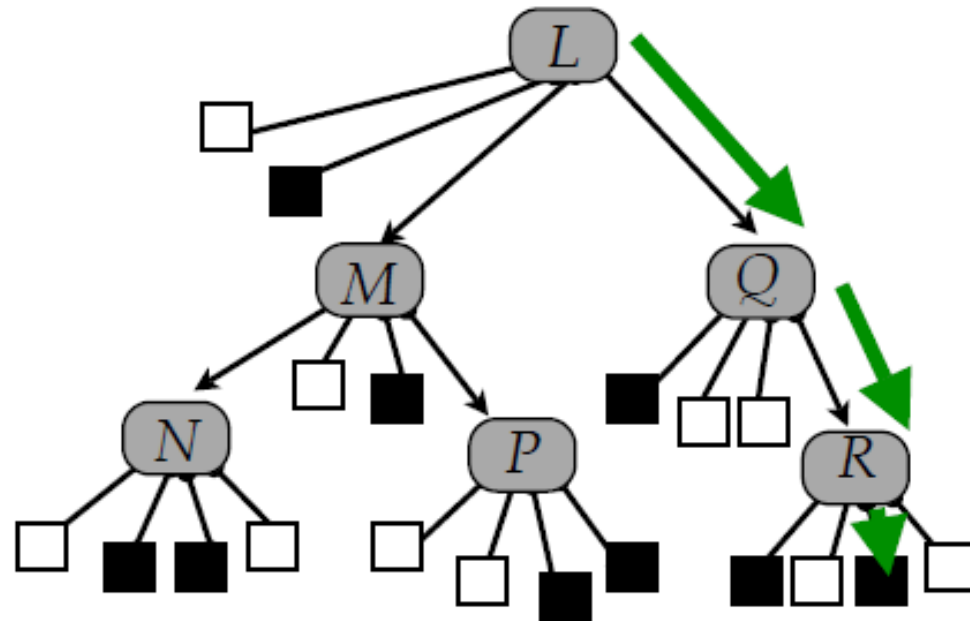
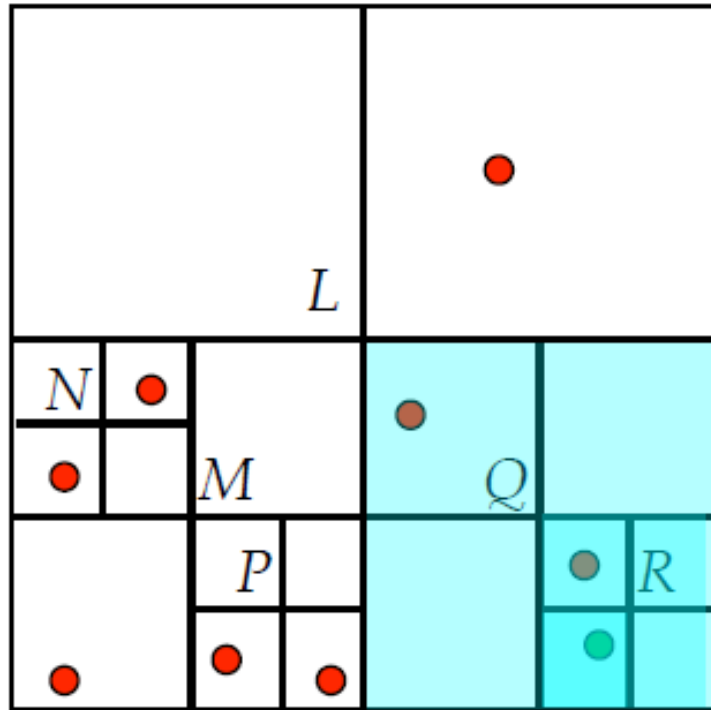


Point-Region quadtrees

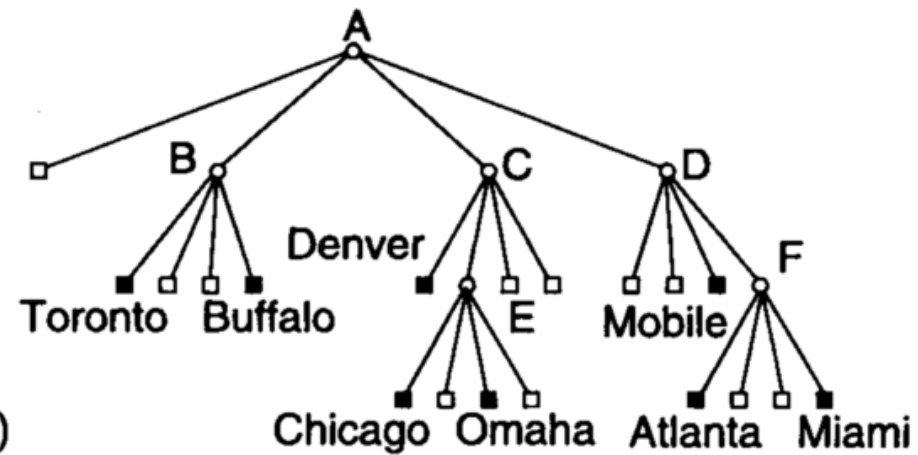
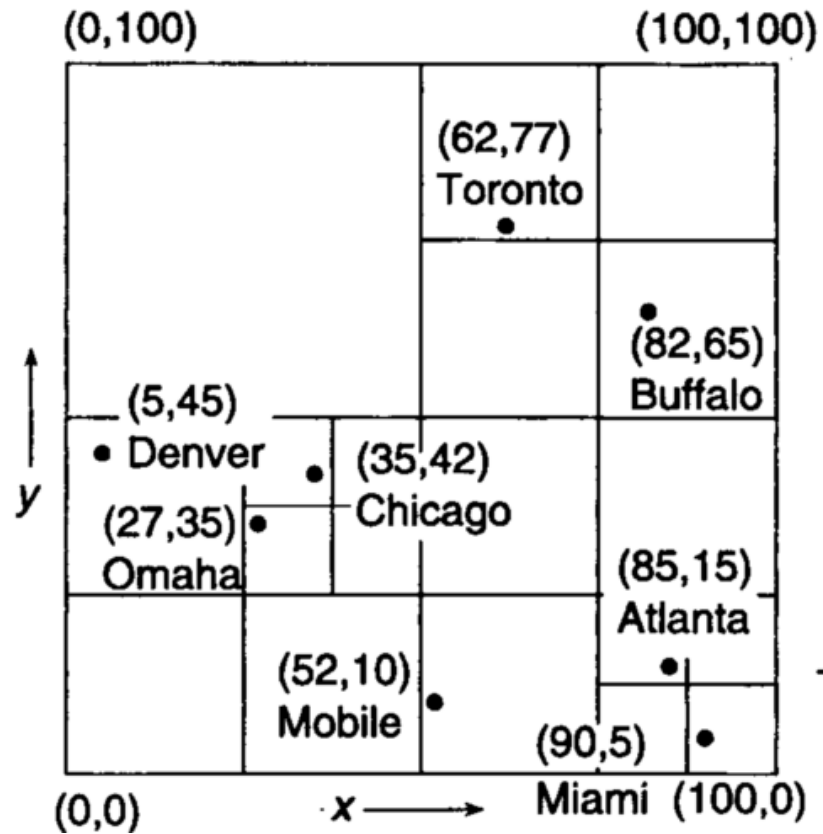


Point-Region quadtrees

- Data queries:



Point-Region quadtrees



PR-Quadtree Range and Window Queries

- ▶ Same traversal as MX-quadtree range search
- ▶ Start at root; for each quadrant, if bounds intersect query rectangle, descend
- ▶ At leaf: test every point in bucket against query (point-in-rectangle or circle)
- ▶ Prune entire subtrees when quadrant MBR disjoint from query region
- ▶ Multiple points per leaf → more point tests but fewer tree levels for clusters
- ▶ Question: Compare cost with k-d tree range search on same dataset.

Point-Region quadtrees

- ▶ **Characteristics:**

- ▶ Domain division doesn't depend on data points
- ▶ All data points locate at leaves
- ▶ The data domain division is done recursively according to the size of the dataset
- ▶ Expansion: There can be more than 1 data points inside one subdomain.

Point-Region quadtrees

- ▶ Disadvantages:

- ▶ Sometimes it is not efficient: if there are multiple points located close together, the data sub-domain has to be too small, resulting in too many data levels
- ▶ Representations for multi-dimensional data domains consume a lot of storage space.

R-Trees

- ❑ Used to store rectangular regions of an image or a map.
- ❑ R-trees are particularly useful in storing very large amounts of data on disk.
- ❑ They provide a convenient way of minimizing the number of disk accesses.



R-Trees — Minimum Bounding Rectangles (MBR)

- ▶ Each node = MBR enclosing all child entries; root MBR covers entire dataset
- ▶ Order k : each node holds $\lceil k/2 \rceil$ to k entries (rectangles or child MBRs)
- ▶ Optimized for disk pages: one node per I/O — minimize page reads
- ▶ Leaf entries: (MBR, record_oid); internal entries: (MBR, child_pointer)
- ▶ Group zones G1,G2,G3; real zones R1–R9 at leaves (slides 79, 80)
- ▶ Contrast with k-d trees: R-trees index regions, not dimension-split points



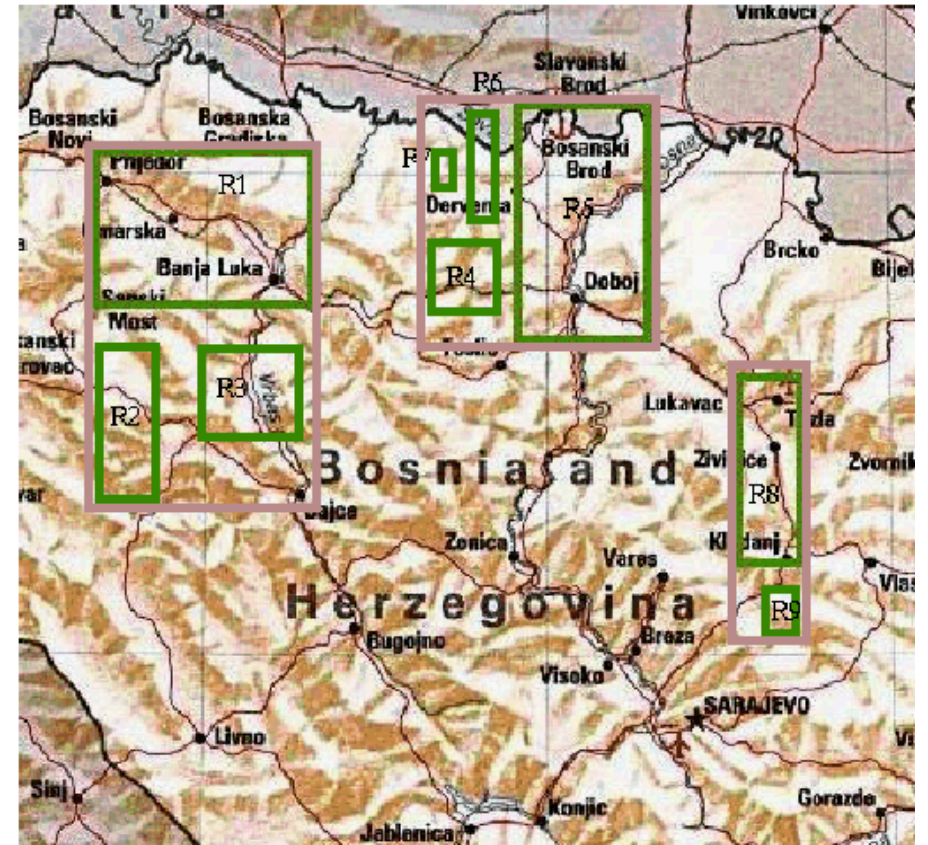
R-trees

► Characteristics

- Each R-tree has an associated order, integer k .
- Each nonleaf R-tree node contains a set of at most k rectangles and at least $\lceil k/2 \rceil$ rectangles.
- Each disk access brings back a page containing at least $\lceil k/2 \rceil$ rectangles.
- Each node is a rectangle with the smallest area containing its child nodes
- All nodes that contain data are leaf nodes.
- Each leaf node contains from $k/2$ (or 2) to k records.

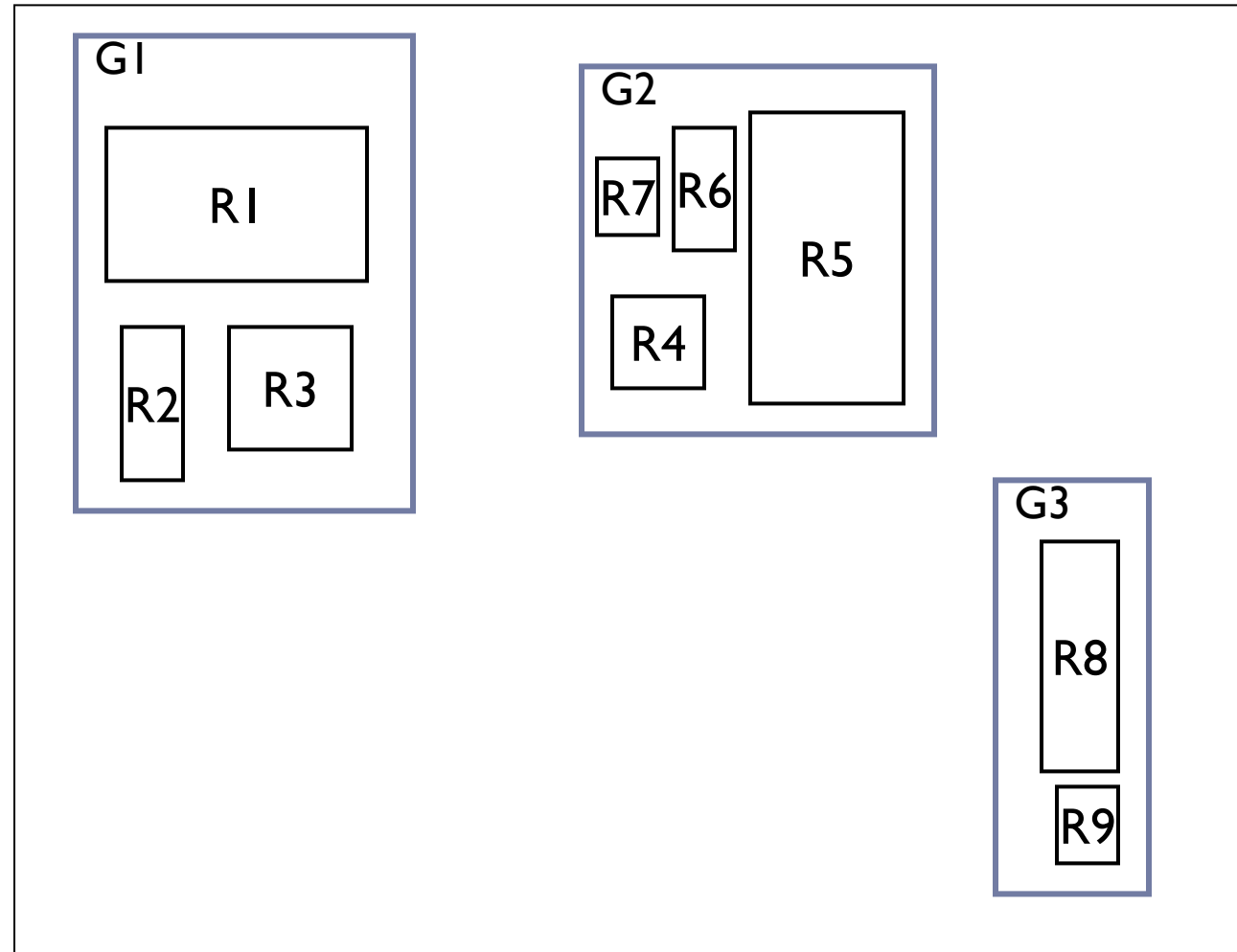
R-trees

- ▶ The R tree contains 2 types of rectangular zones
 - ▶ The "real" zone: contains real data, usually the leaf nodes of the R tree.
 - ▶ Group Zone: are the root nodes of the R tree



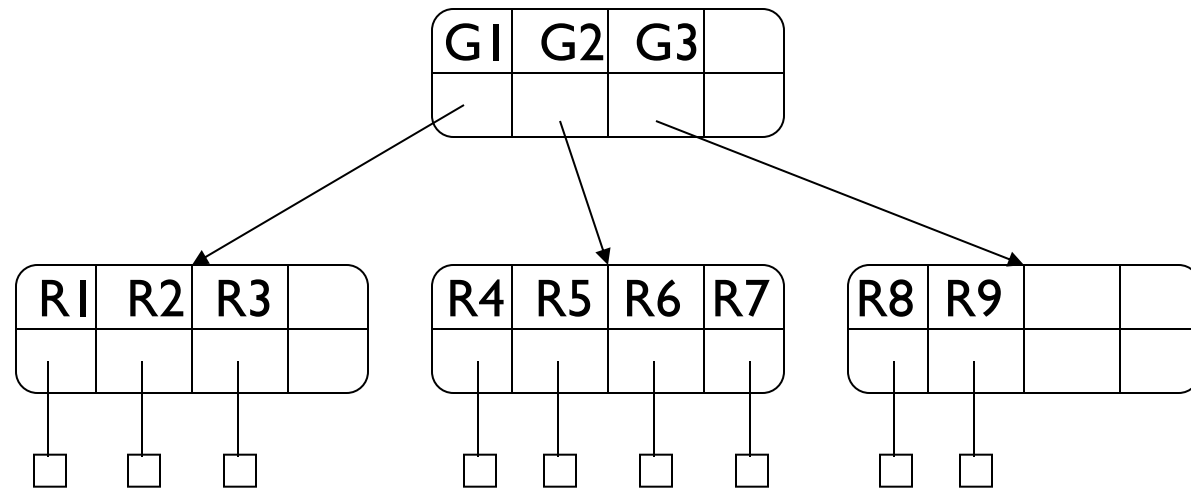
R-trees

► Example



R-trees

► R-Tree of order 4



Data searching on the R-tree

- ▶ The search process is implemented from the root
 - ▶ Select a branch to find
 - ▶ If the data is not found in one branch, switch to another branch.
- ▶ The selection of the search branch is based on the query data

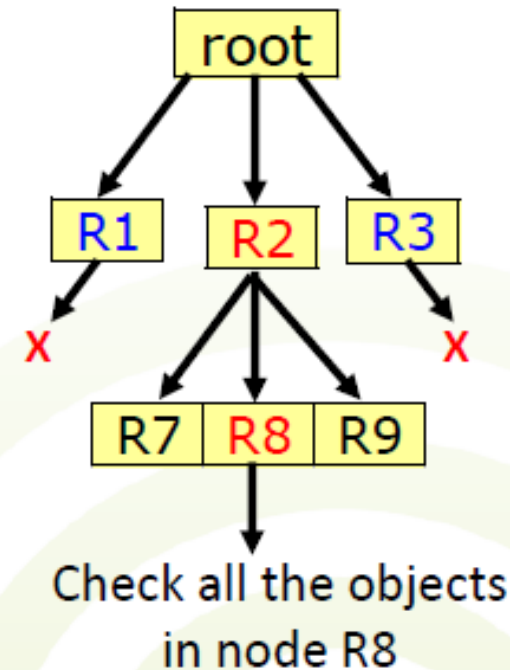
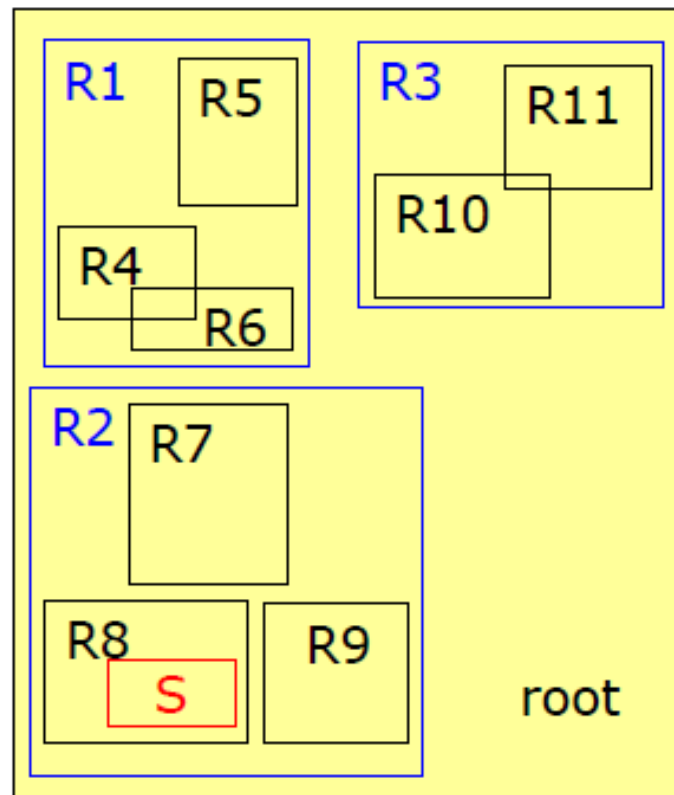
R-Tree Search — Step-by-Step

- ▶ Query region Q (or point q with tolerance for similarity search)
- ▶ From root: for each child MBR, if MBR intersects Q , descend into that child
- ▶ Prune branches whose MBR does not intersect Q
- ▶ At leaf: test each record MBR against Q ; return matches
- ▶ Similarity search: choose branch whose MBR is closest to S
- ▶ Typical $O(\log n)$ page accesses; overlapping MBRs degrade worst case



Data searching on the R-tree

- ▶ Example: Search for data similar to S.

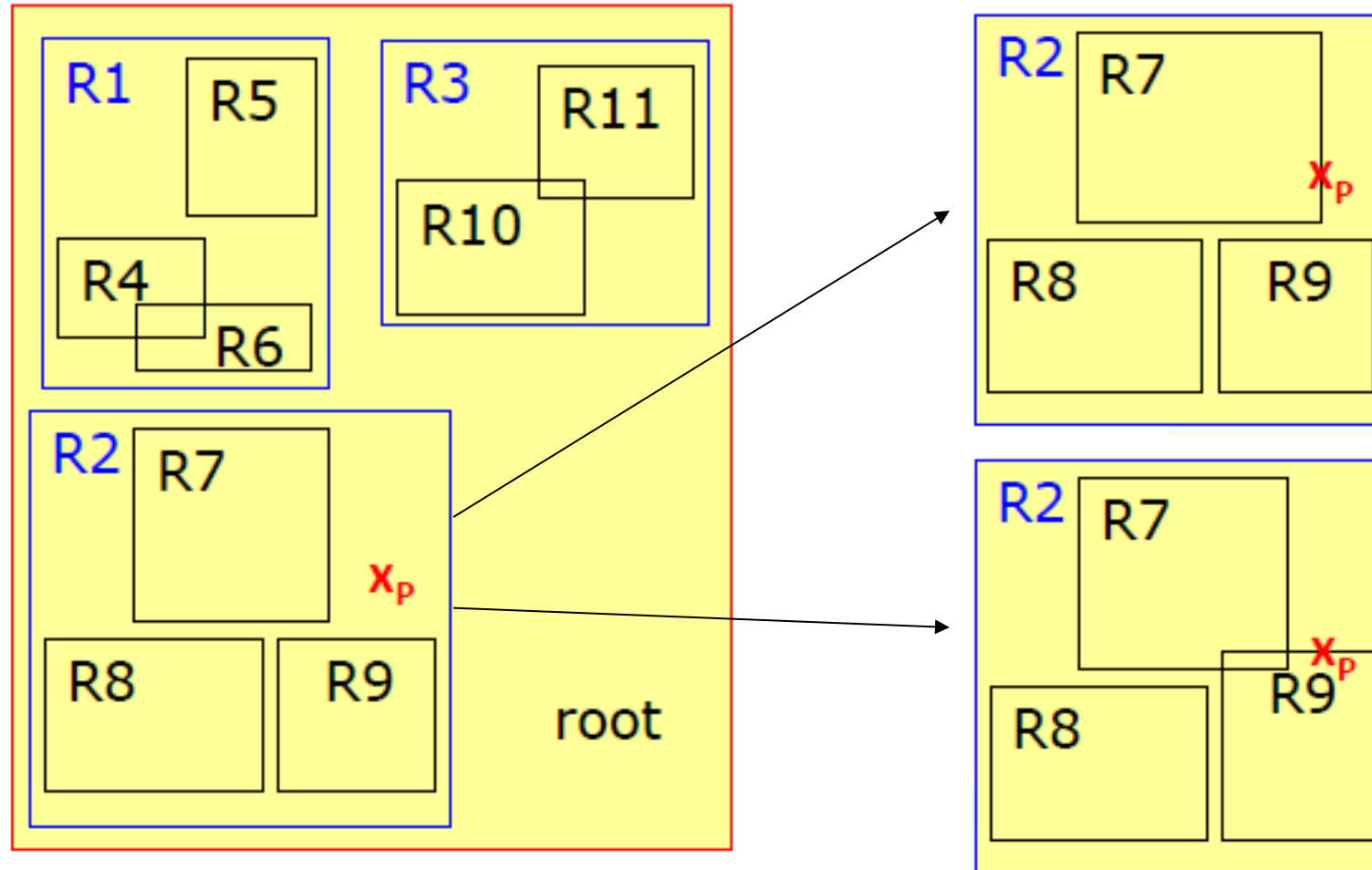


Insertion into a R-Tree

- ▶ Choosing the best leaf node to contain the data to be added.
 - ▶ The best node is the one that needs to expand to the smallest area to accommodate that data.
- ▶ Data will be added to that best node if space is available (the amount of data in that node is currently $< K$)
- ▶ If the amount of data in the best node is full, after adding new data we have to split the node in half.
- ▶ The new nodes (after splitting) will be the leaf nodes.
- ▶ If the root node contains a sufficient number of top nodes, the 2 newly divided nodes will be joined into a new root node.

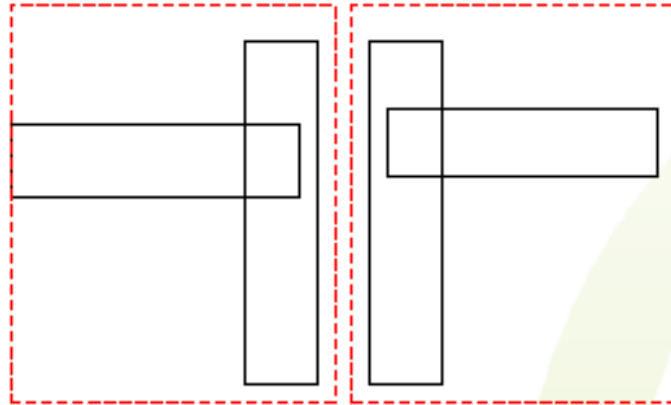
Insertion into a R-Tree

- ▶ Example: 2 options to insert a data point x_p into R-tree.

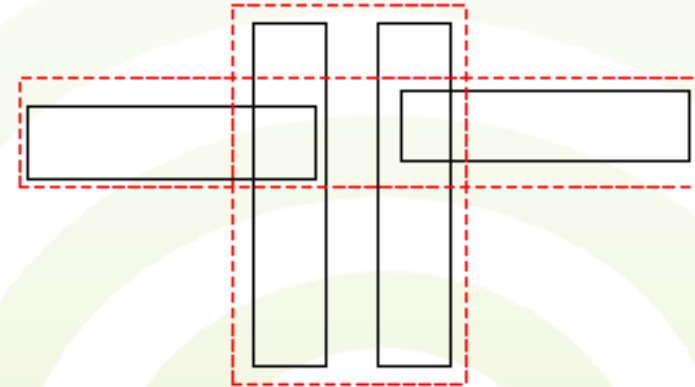


Insertion into a R-Tree

- ▶ In case the node must be split in half:
 - ▶ Minimizing excess space
 - ▶ Minimize overlapping areas



Bad split

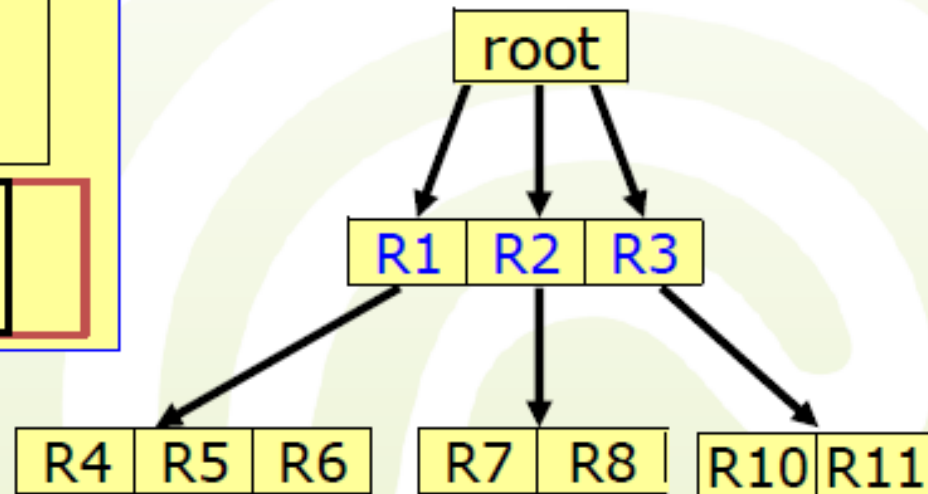
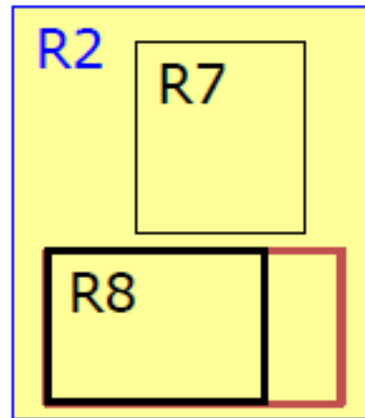
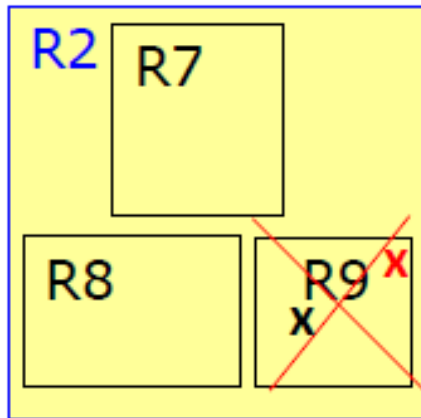


Better Split

- The process of splitting nodes is sometimes complicated.

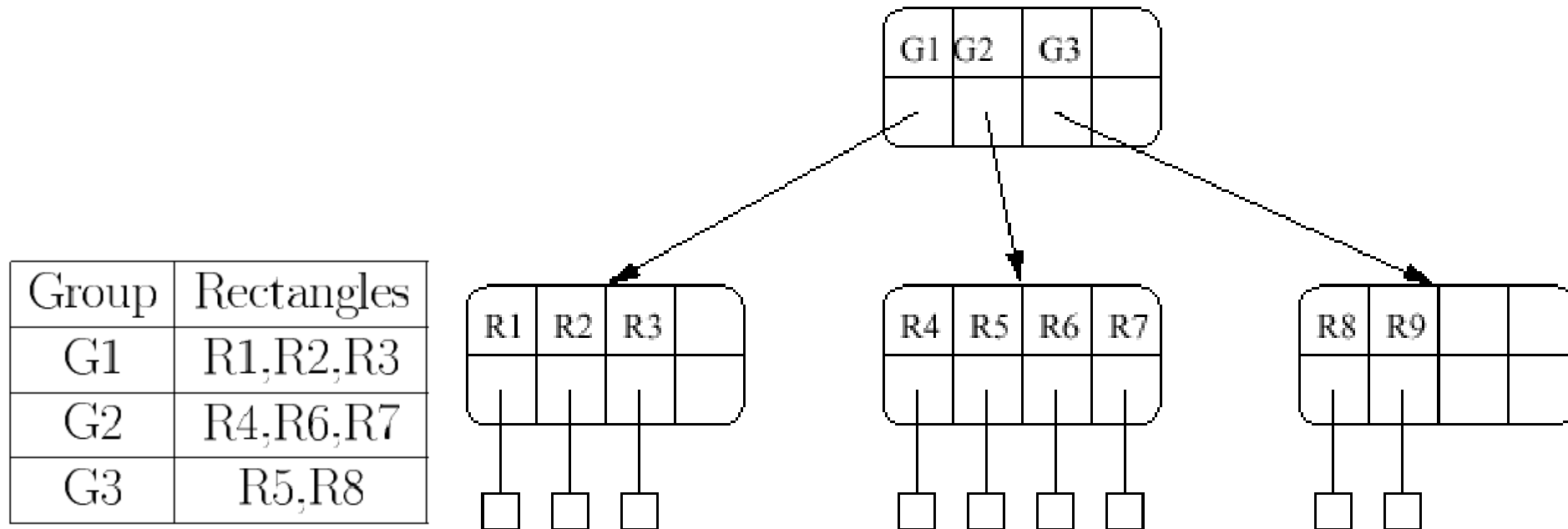
Deletion in R-Trees

- ▶ Find the leaf node that contains the data to be deleted
- ▶ Delete data at that node
- ▶ Deletion of objects from R-trees may cause a node in the R-tree to “underflow” in which contains less than $\lceil k/2 \rceil$ rectangles. We must create a new logical grouping

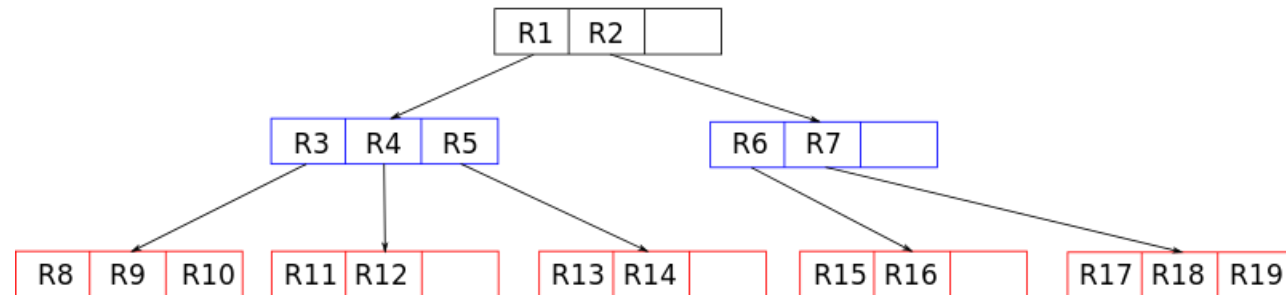
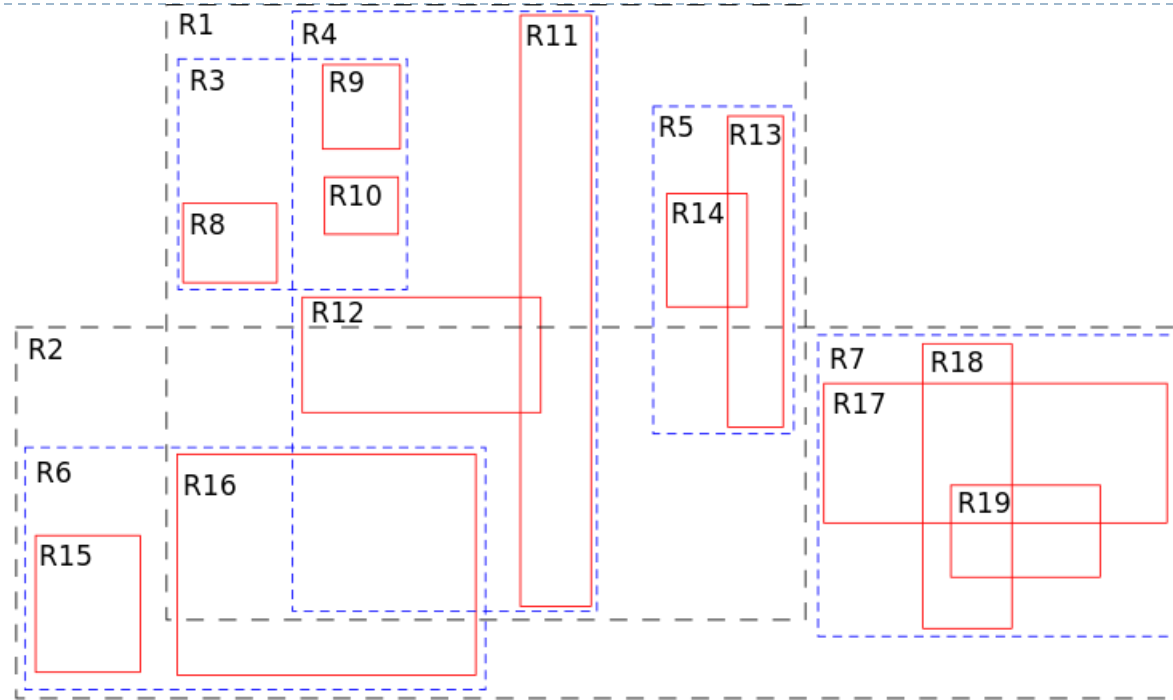


Deletion in R-Trees

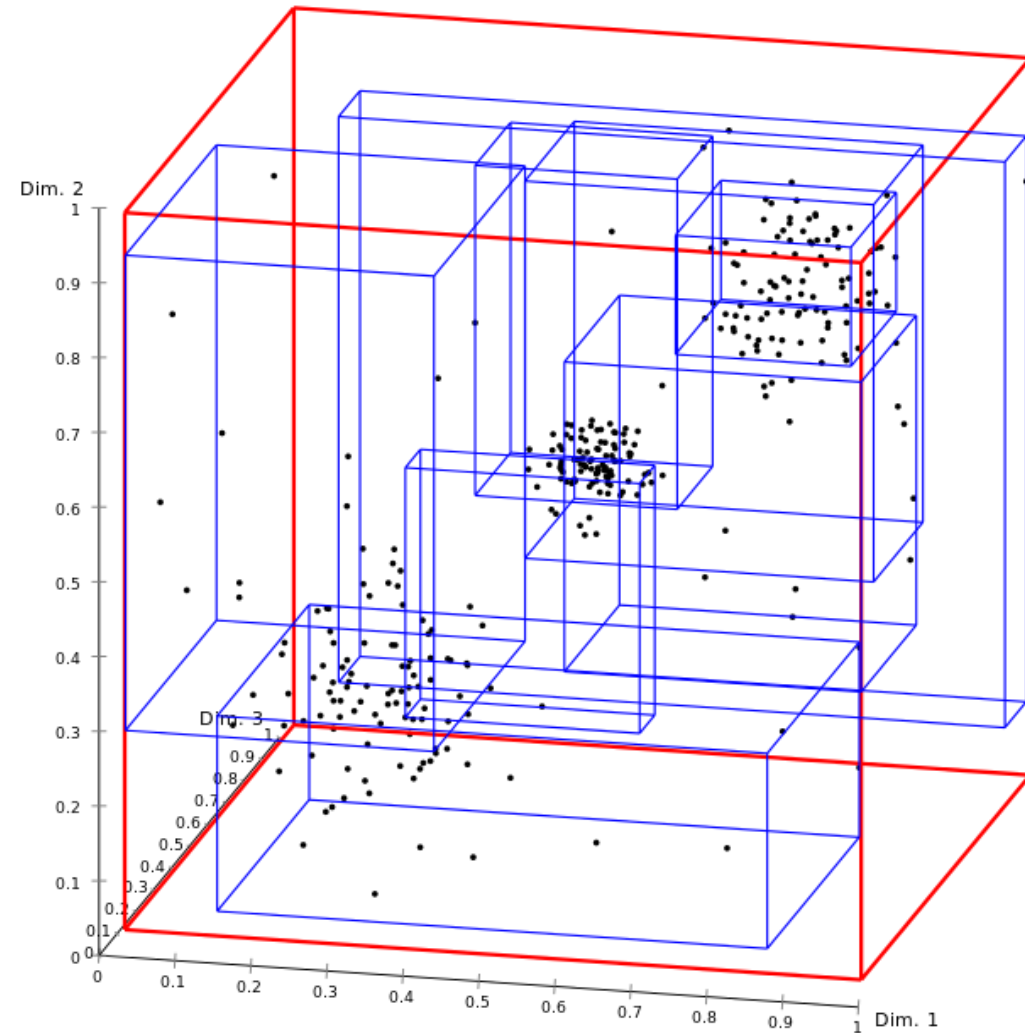
- ▶ In order to ensure the minimum number of regions of each group, when deleting a data node sometimes results in the redivision of the group.
 - ▶ Example: want to delete R9.



R-tree example



R-tree example



Summary — Multidimensional Data Structures

- ▶ k-d trees: binary space partition; best for moderate-dim point data and NN search
- ▶ Point quadrees: 4-ary partition on point coords; intuitive for 2D
- ▶ MX/PR quadrees: grid-based; MX for raster, PR for clustered point buckets
- ▶ R-trees: MBR hierarchy for disk-based region data and GIS
- ▶ Choose structure by: point vs region vs raster, memory vs disk, query type

